



WPI



Life Sciences und
Facility Management

Designing a Creative Coding Course Syllabus

An Interactive Qualifying Project Report Submitted to the Faculty of Worcester Polytechnic
Institute In partial fulfillment of the requirements for the Degree of Bachelor of Science

Submitted on:

October 10, 2024

Submitted by:

Colin Hoar
Artem Iurkovskyi
Ryan Wylie

Sponsored by:

Zurich University of Applied Sciences (ZHAW)

Sponsors:

Dr. Tugçe Bilgin Sonay
Dr. Ahmad Aghaebrahimian
Prof. Dr. Maria Anisimova

Advised by:

Prof. Carolina Ruiz
CFO Emeritus and Instructor Jeffrey Solomon

This report represents the work of one or more WPI undergraduate students submitted to the faculty as evidence of completion of a degree requirement. WPI routinely publishes these reports on the web without editorial or peer review.

Abstract

STEM students may lack opportunities to engage in creative pursuits alongside their technical education. To address this, the Institute of Computational Life Sciences at Zurich University of Applied Sciences sponsored the development of a Creative Coding course. The course integrates coding with artistic expression to foster innovative thinking. The course design process involved evaluating active learning strategies, analyzing existing creative coding courses, and designing a hands-on syllabus. The course includes topics like cellular automata and machine learning that are both familiar to STEM students and encourage creative exploration. By learning creative coding, students are equipped to generate new ideas and perspectives that can inspire innovation within their own technical field.

Executive Summary

Introduction and Background

Individuals who think creatively and solve complex technical problems play a crucial role in advancing human society. STEM students are educated with deep analytical and technological knowledge. However, many STEM students may have little exposure to creative endeavors, particularly those that combine coding with artistic exploration. This gap limits the development of well-rounded problem-solvers who can approach challenges from both technical and innovative perspectives.

To address this, the Institute of Computational Life Sciences at Zurich University of Applied Sciences sponsored this project that focused on the development of a Creative Coding course. A Creative Coding course syllabus was developed to merge programming skills with creative exploration. Creative coding represents a dynamic intersection of artistry and technology, transforming programming from a purely functional tool into a means for artistic expression ([Chibalashvili et al., 2023](#)). Instead of focusing solely on solving applied problems, creative coding encourages students to experiment with creating visuals, sound, and interactive digital content ([Zhang, 2017](#)). This approach allows them to explore how code can be used to generate dynamic graphics, produce engaging soundscapes, and build immersive interactive installations ([Amiri, 2011](#)). Through these projects, students gain hands-on experience in blending artistic creativity with technical skills, which is crucial for developing innovative solutions and broadening their problem-solving abilities.

Creative coding employs a range of tools that illustrate its power to blend technology with artistic exploration. Tools like Processing and p5.js are accessible and popular, enabling users to create visuals and interactive experiences with minimal setup ([Processing Foundation, n.d.](#)). Another tool, Twine, is used for creating interactive stories where users make choices that influence the narrative, adding an element of audience interaction ([Kirilloff, 2021](#)). These tools allow beginners and experts alike to transform code into engaging art, from dynamic graphics to interactive storytelling, showcasing the artistic potential of technology. Examples of visual outputs that can be created using creative coding are shown in Figure ES1.

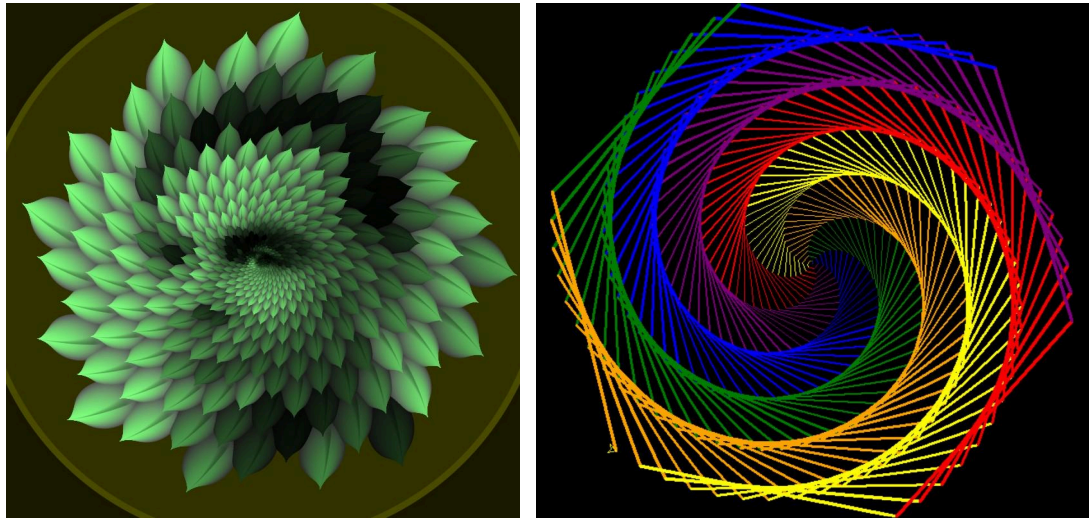


Figure ES1. Examples of Visual Output Created with Creative Coding. (a) Turing morphogenesis ([Kipling, 2013](#)); (b) Archimedes Spiral (Hoar, 2024)

Methodology and Results

Identified and incorporated effective teaching strategies

Five creative coding instructors were interviewed to determine the most effective strategies for teaching creative coding. Active learning, which engages students through activities and discussions rather than passive listening, was encouraged by the sponsor as a preferred teaching strategy. There was significant overlap between this approach and the practices already employed by the interviewed instructors, who incorporate elements of active learning into their teaching methods.

An effective teaching strategy, the use of flipped classrooms was incorporated in the course design, where students study independently before class, leading to improved in-class collaboration. This approach was based on instructors' reports that collaboration enhances learning and creativity. Pre-lecture videos were also introduced to minimize lecture time, allowing for more hands-on activities, as instructors noted that coding skills develop best through practice. Lastly, iterative assignments were used to encourage creativity by focusing on quantity in experimentation, reducing the pressure to achieve perfection, and fostering exploration.

Developed course content and produced course examples

Nine creative coding instructors and research scientists were consulted to develop the course content, determine appropriate tools, and define clear learning goals. The main goals

identified included enabling students to apply the iterative creative process, improving their Python programming skills for creative purposes, and introducing foundational machine learning concepts. The creative process was emphasized to encourage students to think beyond their comfort zones and engage in problem-solving through repeated iterations. Python was selected as the primary programming language due to its accessibility and familiarity among students in STEM fields, despite initial suggestions favoring JavaScript. Due to the growing popularity of artificial intelligence, the incorporation of machine learning as a learning outcome was introduced to enhance the course's appeal and attract greater student interest.

Turtle and Pygame, two Python programming libraries, were chosen to introduce students to creative coding fundamentals in an accessible way. Students begin with Turtle in the first week, using simple tools for generating visuals while gradually tackling more complex outputs. In the following weeks, Pygame is introduced, allowing students to explore basic game development and interactive projects. This progression builds a foundation in visual coding before advancing to interactive applications, ensuring a balanced development of both technical skills and creative exploration.

Conway's Game of Life was incorporated based on its relevance to computational life sciences, as highlighted during one of the interviews. This cellular automaton, which simulates complex patterns through simple rules, was selected for its biological and artistic potential, making it a compelling topic for creative coding exploration. To showcase to students who may be hesitant about taking the course, we have produced a creative output that could be achieved with Game of Life (Figure ES2). In addition, we have completed this course module with Google Colab Notebook which guides the students through the basic rules of the Game of Life, advanced implementations, and creative modifications. As an assignment, students are encouraged to modify the rules to produce unique creative outputs, allowing them to experiment and engage deeply with the iterative process of creative coding.

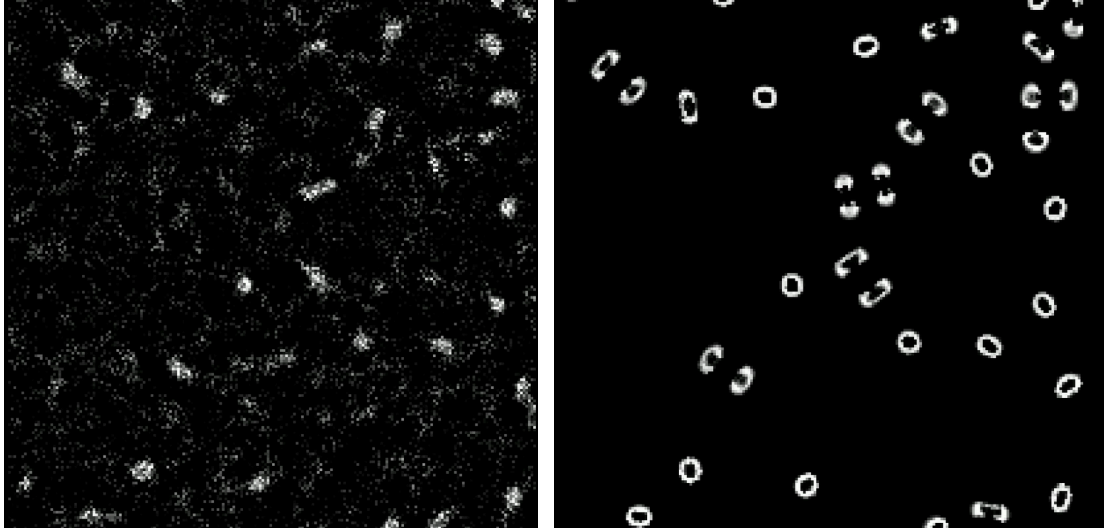


Figure ES2. Generated and Visualized Game of Life as a Project Example. (a) One of the first frames of the simulation, (b) the last frame of the simulation.

Machine learning was introduced as the last topic of the course, in response to the growing widespread use of artificial intelligence. It was considered both a relevant topic for computational life sciences and a powerful tool for generative art. A variety of machine learning tools were evaluated, including Sonic Pi, ml5.js, and OpenFrameworks, but ultimately, ml4a was selected for its Python compatibility and accessibility. Students were introduced to machine learning through guided lectures and hands-on implementation, experimenting with algorithms like ([Glow:Reversible Generative Models](#)) and Deepdream ([DeepDream:hallucinate with a neural network](#)), to create their generative art projects.

Constructed a Creative Coding course syllabus

The course syllabus was constructed based on the information documented in the previous objectives, including the findings from instructor interviews, sponsors and research scientists, and our analysis of course syllabi, as described in each of our project objectives.

Recommendations

Drawing from the course design, the following recommendations have been developed for lecturers to adapt the syllabus

- Choose a programming language that best suits the course and the students. While Python was selected for our syllabus to align with students' existing knowledge, other languages like JavaScript are also suitable as long as they support visualization.
- Select programming applications based on familiarity and usability. We used Google Colab, but any application that facilitates coding and sharing can be used effectively.
- Feel free to adjust course topics. Although we focused on Conway's Game of Life and Machine Learning due to their relevance to STEM students, lecturers are encouraged to experiment with other topics to fit their class needs.

Conclusions

A Creative Coding course was developed, sponsored by the Institute of Computational Life Sciences at Zurich University of Applied Sciences, to integrate coding with artistic expression. The course design drew on insights from existing creative coding courses, applied active learning strategies, and covered engaging topics such as cellular automata and machine learning. These elements were chosen to inspire creative exploration in STEM students while ensuring accessibility. By participating in this course, students develop skills that enhance their creative process and expand their technical expertise

References

- Amiri, F. (2011). Programming as Design: The Role of Programming in Interactive Media Curriculum in Art and Design. *International Journal of Art & Design Education*, 30(2), 200-210. <https://doi.org/10.1111/j.1476-8070.2011.01680.x>
- Chibalashvili, A., Savchuk, I., Olianina, S., Shalinskyi, I., & Korenyuk, Y. (2023). Creative Coding as a Modern Art Tool. *BRAIN. Broad Research in Artificial Intelligence and Neuroscience*, 14(2), 115–127. <https://doi.org/10.18662/brain/14.2/447>
- Kirilloff, G. (2021). Interactive Fiction in the Humanities Classroom: How to Create Interactive Text Games Using Twine. *The Programming Historian*, 10(10). <https://doi.org/10.46430/phen0095>

Kogan, G. (2020). *Glow model example* [Jupyter notebook]. GitHub.

<https://github.com/ml4a/ml4a/blob/master/examples/models/glow.ipynb>

Kogan, G. (2021). DeepDream model example [Jupyter notebook]. GitHub.

<https://github.com/ml4a/ml4a/blob/master/examples/models/deepdream.ipynb>

Processing Foundation. (n.d.-a). *Education*. <https://processingfoundation.org/education>

Processing Foundation. (n.d.-b). *Libraries*. <https://p5js.org/libraries/>

Zhang, L. (2017). *Foundations of research in creative computing*.

<https://doi.org/10.17870/bathspa.00011588>

Table of Contents

Abstract.....	i
Executive Summary.....	ii
Table of Contents.....	viii
List of Figures.....	x
List of Tables.....	xi
Authorship.....	xii
Acknowledgments.....	xiii
1. Introduction.....	1
2. Background.....	3
2.1 Stakeholders.....	3
2.2 Site and Sponsors.....	4
2.3 Definition of Creative Coding.....	4
2.4 Tools for Creative Coding.....	5
2.4.1 Interactive Fiction and Storytelling as a Specialized Product.....	6
2.4.2 Context-Free Grammars as a Domain-Specific Language for Procedural.....	7
2.5 Active Learning as a Teaching Strategy.....	8
2.6 Case Studies.....	10
2.6.1 Primary School Creative Coding.....	10
2.6.2 VA345 Creative Coding, Sabanci University.....	10
2.6.3 Takeaways.....	11
2.7 Summary.....	11
3. Methodology.....	13
3.1 Objective 1: Evaluate the Effectiveness of Different Teaching Strategies in the Context of Creative Coding.....	13
3.1.1 Semi-Structured Interviews with Course Instructors.....	13
3.1.2 Survey with Students About the Learning Strategies They Find Effective.....	14
3.2 Objective 2: Identify and Analyze the Structures of a Sample of Existing Creative Coding Courses.....	14
3.2.1 Semi-Structured Interviews with Creative Coding Experts.....	15
3.2.2 Obtain and Analyze Creative Coding Course Syllabi.....	16
3.3 Objective 3: Construct our Creative Coding Course Syllabus.....	16
3.3.1 Process for Making the Syllabus.....	16
3.3.2 Develop Familiar Topics to Serve as Exemplars for Students.....	17
3.4 Objective 4: Evaluate the Interest and Effectiveness of the Syllabus.....	17
3.4.1 Interest.....	18
3.4.2 Effectiveness.....	18

3.5 Summary.....	18
4. Results and Discussion.....	19
4.1 Teaching Strategies Results.....	20
4.1.1 Student Collaboration Improves Learning.....	20
4.1.2 Effective Instruction Techniques.....	20
4.1.3 Creativity-Specific Teaching Strategies.....	21
4.2 Course Goals, Audience, and Content Results.....	22
4.2.1 Course Goals Results.....	22
4.2.2 Tools, Resources, and Audience Results.....	23
4.3 The Course Syllabus.....	28
4.4 Interest and Effectiveness of the Syllabus.....	34
4.4.1 Interest.....	34
4.4.2 Effectiveness.....	34
5. Conclusions and Recommendations.....	36
5.1 Conclusions.....	36
5.2 Recommendations.....	37
5.2.1 Recommendations for Lecturers Who Use Our Course Syllabus.....	37
5.2.2 Recommendations for Future Work on the Project.....	38
References.....	39
Appendix A. Creative Coding Interview Consent Agreement.....	45
Appendix B. Teaching Strategy Interview Consent Agreement.....	48
Appendix C. Student Survey.....	51
Appendix D. Effectiveness Interview Consent Agreement.....	53
Appendix E. Google Colab Assignment.....	56
Appendix F. Interview Codebook.....	62

List of Figures

Figure ES1. Examples of Visual Output Created with Creative Coding.....	iii
Figure ES2. Generated and Visualized Game of Life as a Project Example.....	v
Figure 1. Duel, generated with Game of Life.....	25
Figure 2. Ccnwwys, generated with Game of Life.....	26
Figure 3. Generated and visualized Game of Life with MNCA rules.....	27
Figure E.1: Game of Life; Google Colab Notebook Snippet 1.....	57
Figure E.2: Game of Life; Google Colab Notebook Snippet 2.....	58
Figure E.3: Game of Life; Google Colab Notebook Snippet 3.....	59
Figure E.4: Game of Life; Google Colab Notebook Snippet 4.....	60
Figure E.5: Game of Life; Google Colab Notebook Snippet 5.....	61

List of Tables

Table 1. The Creative Coding Course Syllabus.....	29
Table 2. The Course Schedule.....	30

Authorship

Section	Author(s)	Editor(s)
Abstract	Artem I	All
Executive Summary	Artem I	All
Introduction	Colin H	All
Background	All	All
Methodology	All	All
Results and Discussion	All	All
Conclusions and Recommendations	Colin H, Ryan W	All
Appendices	All	All

With the permission of our project advisors, we used ChatGPT as a generative AI writing assistant tool to smooth certain parts of the narrative in this document, including grammatical and stylistic improvements. We attest that all the project content, including but not limited to the literature review, methodology, data gathering, results, analysis of results, conclusions, and recommendations, is our own original content.

Agatha Zeren drafted the first version of Section 2.4 but otherwise was not involved in writing, editing, or reviewing the final report.

Acknowledgments

We would like to express our sincere gratitude to our sponsoring organization, the Zurich University of Applied Sciences, for providing us with the opportunity to work on the development of our Creative Coding Course Syllabus. We extend our heartfelt thanks to our sponsors, Dr. Tugçe Bilgin Sonay, Dr. Ahmad Aghaebrahimian, and Prof. Dr. Maria Anisimova, for their continuous support and availability throughout the project. Their guidance and insightful feedback greatly contributed to the success of this project.

We are also deeply appreciative of our advisors, Prof. Carolina Ruiz and CFO Emeritus and Instructor Jeffrey Solomon, for their invaluable advice and expertise. Their input made the project process smoother and more efficient.

We would also like to extend our sincere appreciation to Agatha Zeren for her contribution to the early development of the project and for formulating the methodology.

This project would not have been possible without the combined support of our sponsors and advisors, and we are truly grateful for their contributions.

1. Introduction

In an era marked by technological advancement and innovation, the demand for individuals who can think creatively and solve complex problems is paramount. At Zurich University of Applied Sciences (ZHAW), students excel in fields such as science and engineering, demonstrating a strong foundation in technological knowledge and analytical skills.

However, many STEM students may lack exposure to creative endeavors such as coding for artistic expression or interactive design. Recognizing the importance of nurturing creativity alongside technical proficiency ([Bequette & Bequette, 2012](#)), there is a pressing need to develop interdisciplinary courses that introduce students to the world of creative coding. By merging principles of coding with creative exploration, these courses would broaden students' perspectives, foster innovation, and empower them to apply their technical skills in novel and imaginative ways.

The primary goal of this project was to design a Creative Coding course syllabus tailored to the unique needs and interests of students and professors at ZHAW. This course not only introduces students to the fundamentals of coding for creative purposes but also incorporates active learning pedagogy to engage and inspire learners. The resulting course adopted a continuing education format, comprising 3 European Credit Transfer and Accumulation System (ECTS) credits. By bridging the gap between technology and creativity, the course seeks to equip students with the tools and mindset necessary to become versatile problem-solvers and innovators in their respective fields.

To achieve this goal, we determined four research objectives. The first objective was to evaluate the effectiveness of different teaching strategies in the context of creative coding, considering factors such as student engagement, learning outcomes, and accessibility. The second objective was to identify and analyze the structures of a sample of existing creative coding courses. The first two objectives informed the third: the construction of a syllabus for the Creative Coding course. The course draws upon existing courses and pedagogical frameworks from various schools around the world while tailoring content to the specific needs and interests of ZHAW students. The final objective was to implement mechanisms to measure student interest and assess the effectiveness of the syllabus we created.

By combining interdisciplinary material with active learning pedagogies, our course syllabus not only bridges the gap between technology and creativity but also paves the way for

future curricula that foster versatile, innovative thinkers equipped to tackle complex problems in a rapidly evolving world. This approach to course development has the potential to inspire other institutions to adopt similar interdisciplinary frameworks, ultimately fostering a new generation of creative problem-solvers.

2. Background

In this section, we outline existing research on creative coding and provide context for our course development. First, we describe the key stakeholders, the sponsoring institution, and the site of the course. Then, we define creative coding and identify important topics within the field. Next, we evaluate teaching strategies and specific tools used in creative coding and review examples of creative coding courses to frame our approach. From there, we review examples of existing creative coding courses from the literature to contextualize our course.

2.1 Stakeholders

This creative coding course was designed to appeal to a very diverse range of individuals across various fields, each with unique interests and motivations. Educators, for example, may be interested in incorporating this creative coding course into their curriculum to enhance students' technical skills while fostering creativity and collaboration. Collaborative projects encourage students to work together, blending diverse perspectives and skills to solve problems and create innovative work. The course offers instructors an opportunity to diversify their teaching methods and cater to the growing demand for interdisciplinary learning ([Ivanitskaya, Clark, Montgomery, & Primeau, 2002](#)). For computer science students, this course provides a practical application of coding concepts beyond traditional software development. Meanwhile, STEM students can explore interactive and generative art through the familiar medium of coding, expanding their creativity while deepening their understanding of technology.

Innovators and entrepreneurs see creative coding as a tool for prototyping ideas and developing interactive products. Whether designing immersive experiences for virtual reality or creating data-driven installations for marketing purposes, a foundation in creative coding can empower entrepreneurs to bring their visions to life. Even those outside formal educational or professional settings would be drawn to creative coding for personal enrichment and enjoyment. Hobbyists engage in coding projects as a means of self-expression, problem-solving, and exploration, finding fulfillment in the process of creating something unique and meaningful. The appeal of the creative coding course extends across academia, the arts, entrepreneurship, and personal interests, offering a diverse range of individuals the opportunity to enhance their skills, foster creativity, and explore new possibilities at the intersection of technology and expression.

2.2 Site and Sponsors

The Creative Coding project is sponsored by The School of Life Sciences and Facility Management (LSFM) at ZHAW, Zurich University of Applied Sciences. Attended by more than 10,000 students, ZHAW offers 34 bachelor's and 20 master's programs. Our direct sponsors are faculty at the Institute of Computational Life Sciences within ZHAW's LSFM based in Wädenswil, Switzerland. The creative coding course offering reflects their vision for innovative research and teaching methods. Computational Life Sciences is an emerging field that takes advantage of computer science capabilities to make breakthroughs in biology and the life sciences. The introduction of a creative coding curriculum aims to redefine the role of computer science, positioning it as a versatile tool extending beyond its traditional confines. This paradigm shift in viewing computer science is anticipated to bring substantial benefits to the sponsor.

2.3 Definition of Creative Coding

Creative coding represents a dynamic intersection of artistry and technology, where programming is not merely a tool for functional tasks but a means of artistic expression. This innovative field has gained traction in recent years, propelled by the integration of science, technology, engineering, art, and mathematics (STEAM) education ([Chibalashvili et al., 2023](#)). Creative coding transcends traditional programming models, aiming to implement artistic ideas through the creation of visual, auditory, or interactive experiences using code. It differs from traditional programming, which focuses on solving specific applied tasks, by its intuitive approach to the programming process ([Zhang, 2017](#)).

At its core, creative coding can serve as a tool for imaginative exploration across various media forms. Through the manipulation of software code, artists can translate data from one medium to another, transforming dry statistical information into engaging creative projects. This versatility allows for the integration of text, sound, image, video, 3D models, and more into interactive art installations, multimedia performances, or immersive experiences ([Amiri, 2011](#)). The reach of creative coding extends beyond typical artistic mediums, encompassing interdisciplinary art projects that challenge traditional boundaries. By engaging with programming languages such as Processing (one of the most popular creative programming tools) ([Processing Foundation, n.d.-a](#)), artists can explore different approaches to digital art and

design, utilizing intuitive programming processes to bring their creative visions to life ([Chibalashvili et al., 2023](#)). This bridge between technology and creative expression leads to experimentation and innovation, enabling artists to push the boundaries of traditional artistic practices.

One of the distinguishing features of creative coding is its ability to alter the creative process, offering accessible tools and resources for individuals of many different experience levels to engage with programming and digital art. This inclusive approach reflects a broader shift in the perception of programming as a form of creative expression, challenging traditional engineering-based metaphors and methodologies ([Artut, 2017](#)). Furthermore, creative coding encourages collaboration and interdisciplinary exchange, bringing together artists, designers, programmers, and technologists to collaborate on innovative projects. This collaborative climate mirrors the spirit of STEAM education, which emphasizes the integration of diverse disciplines to foster creativity and innovation from an early age ([Chibalashvili et al., 2023](#)). In the realm of creative coding, the possibilities are limitless. Artists are empowered to experiment with new technologies, explore uncommon mediums, and challenge ordinary notions of art and technology. Whether it is through generative audio-visuals, computational art, or interactive installations, creative coding offers a playground for artistic experimentation and exploration ([Yang & Zhang, 2016](#)).

To summarize, creative coding represents a dynamic and evolving method of creative expression, where programming serves as a creative tool for exploring new frontiers in digital art and design. Through the combination of arts and technology, creative coders push the boundaries of traditional artistic practices, fostering collaboration, innovation, and accessibility in the creative process ([Chibalashvili et al., 2023](#)).

2.4 Tools for Creative Coding

The broad scope for creative coding comes with an equally expansive set of tools used for it, especially considering specialized or custom tools. While some of these tools are created for personal comfort or as a learning exercise, many are created to fill a specialized need either for input language or output medium. Most general-purpose tools focus on the format of a graphics window, with support for general-purpose drawing, interactivity, and basic sound output capabilities. For more experimental or uncommon outputs, such as human dancers, show lighting

control, or interactive fiction, specialized or custom tooling can fill the gap ([Chibalashvili et al., 2023](#)). Alternatively, such tooling can be created in order to provide a different structure for writing code, allowing artists to express themselves in a different way.

Presented here is a small selection of general- and special-purpose tools to illustrate these ideas, without attempting to be exhaustive or representative of all facets of the field.

Processing is a long-standing general-purpose creative coding tool. It is a programming language and web-based Integrated Development Environment (IDE) for creative coding that dates back to 2001, and, along with the accompanying documentation, is widely used in independent and educational settings ([Processing Foundation, n.d.](#)). Processing software, for the most part, controls a fixed-size canvas with a simple drawing API and also supports sound and interactivity. The documentation around Processing focuses on making it a low barrier of entry for both students and teachers, providing extensive documentation and resources targeted at educators ([Processing Foundation, n.d.-a](#)).

p5.js is an implementation of a Processing-inspired application programming interface (API) as a library for JavaScript, rather than as an independent programming language, and is also stewarded by the Processing Foundation. It hosts a large suite of 3rd party libraries for extending its functionality ([Processing Foundation, n.d.-b](#)).

2.4.1 Interactive Fiction and Storytelling as a Specialized Product

One way creative coding tools are specialized is to output a different medium. Various media are represented in creative coding works, from lighting to human dancers ([Chibalashvili et al., 2023](#)). One medium in this category seen in educational contexts is interactive narrative. This textual form uses a variety of computational tools to construct a story, influenced by interaction from the reader. One of the simplest incarnations of this is in gamebooks, such as the Choose Your Own Adventure ([Choose Your Own Adventure, n.d.](#)) or Fighting Fantasy ([Fighting Fantasy, n.d.](#)) franchises. Many of these works, being printed books, ask the reader to also take up the computation in the narrative, aided by dice or other random number generation and memory in a character sheet. Digital works, however, can make use of much greater complexity in computation and can include explicitly written and procedurally generated text interspersed with each other, as well as sophisticated mechanisms for tracking the state of the story and composing scenes together.

Ink is a markup and programming language used for writing interactive stories with list-of-options choice points ([Inkle Studios, n.d.](#)). It excels at dialog, and can also be used for dynamically or semi-dynamically assembling smaller snippets of story together based on the state of the world ([Inkle Studios, n.d.](#)). In addition to the built-in features for storing the state of the world, Ink is designed to be easily embedded inside a larger project and integrated into its existing structures ([Inkle Studios, 2022](#)). Ink is bundled with an IDE, Inky, which allows for simultaneous play and authorship of the story, as well as tooling for styleable web exports, which provides an approachable platform for an educational context ([Cardona-Rivera et al., 2019](#); [Inkle Studios, n.d.](#)).

Twine is a tool for interactive storytelling based on hypertext. In addition to navigating via a traditional list of options, hyperlinks can be interspersed throughout the text, allowing more freedom of structure for stories. This can be used for a poetic mutating text, for presenting interactive objects within a description of an environment, or for other structures. Twine also boasts a very visual web-based IDE, significantly impacting its popularity in educational environments, especially for non-programmers ([Kirilloff, 2021](#)).

2.4.2 Context-Free Grammars as a Domain-Specific Language for Procedural

The first-generation method of creative coding was procedural generation. Procedural generation refers to writing a series of rules that shape a randomized process to create a final result. Any programming language can be used to implement procedural generation, but one can also use tools that make it easier to use rules in a particular structure. One such structure of rules for procedural generation is Context Free Grammar (CFG), a tool for analyzing languages developed for the study of linguistics ([Bohrer, 2023](#)). In software development, CFGs are used to parse the structure of programming languages, but they were originally developed from the perspective of generating sentences, and procedural generation tools use them in a similar way ([Bohrer, 2023](#)). By using the theoretical tools of computer science, these projects have found a different way to think about and create the structure of art. This is an example of special-purpose creative coding tools that are specialized to an input language, rather than an unconventional output.

Tracery is a JavaScript library that implements an embedded domain-specific language for specifying a context-free grammar that generates medium-length texts ([Compton et al.,](#)

[2015](#)). Tracery writers specify how hand-written text fragments are composed together to create texts, examples of which can be generated randomly. The close integration with JavaScript also allows writers to extend the generation beyond pure context-free grammar ([Compton, n.d.](#)). Context-free art is a tool that extends context-free grammar from generating a linear string to a two-dimensional image ([John Horrigan, 2019](#)). With relatively simple rules, detailed scenes can be created by the fractal composition of colored triangles, circles, and rectangles ([John Horrigan, 2019; kipling, n.d.](#)).

2.5 Active Learning as a Teaching Strategy

Traditional teaching methods, which prioritize well-defined and standalone assignments, have proven effective for computer science classes that focus on solving optimization problems, paving the way for a foundational understanding of the subject. However, aligning coursework with real-world industry requirements becomes more challenging. Specifically, integrating less restricted projects into the curriculum is becoming harder within the traditional pedagogical approaches ([Kejriwal, 2022](#)). To implement those techniques comprehensively, more computer science courses employ Active Learning methodology to allow students to define problems, find solutions, and communicate results independently of instructors ([Vernon & Blake, 1993](#)).

As defined by Freeman et al. (2014, p. 8413), “Active learning engages students in the process of learning through activities and/or discussion in class, as opposed to passively listening to an expert” ([Freeman et al., n.d.](#)). By changing the practices of course delivery, active learning techniques improve the engagement of students in the classroom and yield higher retention rates in computer science programs. On top of that, it forces students to pursue information actively and independently. This idea is so fundamental to human cognition that it has inspired the creation of machine-learning algorithms that mimic human learning behavior ([Tran et al., 2018](#)).

While positively affecting the teaching modes in established courses, Active Learning also accelerates the development of a new branch of computer science, creative coding, that does not focus on predetermined outcomes. This is why creative coding, where uncertain outcomes are encouraged by design, is almost naturally aligned with active learning practices. Anderson et al. (2011, p. 1852) document the design of the creative coding course using the Activity-Led Learning method, a version of Active Learning ([Anderson et al., 2012](#)). By dividing the course into 6 one-week creative projects that do not require a lot of background knowledge, the

instructors built an activity-centered approach with lectures serving as supporting material. This approach was reported to benefit students' engagement and interest in exploring the subject further ([Vernon & Blake, 1993](#)).

Central to the implementation of active learning practices, the importance of the role of instructors in course delivery cannot be overstated. While the independent nature of active learning diminishes the traditional importance of a lecturer, instructors assume other roles that tend to be more helpful to students ([Freeman et al., n.d.](#)). The role of a facilitator is believed to be the most effective for instructors to assume. This is where educators can monitor the progress of the group and invest time into giving tailored feedback or advice on project-specific challenges.

In non-project-centered courses, active learning is usually applied through flipped classroom techniques. Students study material before class and instructors allocate most of the time in class to addressing students' misconceptions. Discussions in small groups are also encouraged as part of the flipped classroom approach. Introduced by Caceffo et al. (2018, p. 922), the flipped classroom improved student motivation and final evaluation by 30% and 16% respectively ([Caceffo et al., 2018](#)). This overall shift of an instructor's role from a lecturer to a tailored feedback giver and advisor presents great challenges as well as improved learning outcomes ([Liao & Ringler, 2023](#)).

Traditional teaching methods often yield predictable outcomes. Met with resistance to altering established course delivery techniques, many course instructors are unwilling to redesign already established courses. At the same time, Liao and Ringler argue that active learning should be integrated into pedagogy slowly. The authors found that instructors often face an inability to measure the effect of each aspect of active learning introduced into the classroom ([Liao & Ringler, 2023](#)). The challenge is coupled with students being "scared away" by encountering a teaching setting they have never seen before. Anderson et al. (2011, p. 1852) put forward the idea that students may perceive the guidance within active learning as "scaffolding" and intrusion in their personal space ([Anderson et al., 2012](#)). Active learning and similar techniques may be disproportionately beneficial to active students as opposed to passive students who do not benefit from interactions while learning ([Eickholt, 2018](#)).

To summarize, Active Learning, through its diverse applications, has demonstrated significant benefits, enhancing both the role of the instructor and the engagement of students. It

therefore should be thoughtfully integrated with traditional methods to fully realize its potential in designing a creative coding course.

2.6 Case Studies

In this section, we highlight specific examples of creative coding courses that currently exist or have been taught in the past. These courses detail specific aspects of what is taught and the teaching strategies for teaching students creative coding. They also detail the learning objectives and purpose of the course for each study.

2.6.1 Primary School Creative Coding

The article, “Problem solved, but how? An exploratory study into students’ problem-solving processes in creative coding tasks,” by Karen Woo and Garry Falloon, discusses how creative coding is an efficient way of teaching children in primary and secondary school necessary computer skills and problem-solving through easy to understand block code and exciting visuals ([Woo & Falloon, 2022](#)). The researchers conducted semi-structured interviews and other observations gathered by the teachers to analyze the effectiveness of using creative coding to grow problem-solving skills. They found that students who engaged in creative coding tasks showed significant improvements in their ability to approach and resolve complex problems, demonstrating enhanced critical thinking and creativity. However, this course serves as a more introductory level towards computational skills and problem-solving for a much younger demographic, in contrast to our course, which targets college-level students who may already possess these skills. From this article, we can gain valuable insights into effectively teaching both creative coding and fostering creativity.

2.6.2 VA345 Creative Coding, Sabanci University

VA345 Creative Coding is an undergraduate art class at Sabanci University, Istanbul, Turkey. The article “Incorporation of computational creativity in arts education: Creative coding as an art course” by Selcuk Artut details an art course for students to learn computational skills and discover works of actual computational artists ([Artut, 2017](#)). The article also discusses why art students should take VA345 Creative Coding. The article cites that students need to learn in interdisciplinary courses, specifically creativity and computer skills, which are in high demand.

The course is very introductory to programming, so students only use a computer in week four. It is specifically designed for art students to teach code in a creative, familiar way. Our potential students, however, are different for our course, as we created a course for STEM students. The students we created the course for are already familiar with coding or at least possess the skills to think computationally and learn how to think creatively.

Something that both VA345 Creative Coding and our course accomplish is to introduce different topics under creative coding to the students. VA345 outlines the week-by-week schedule for the expression of creative coding that is being focused on that week. Each week, the students complete a small art project, which they display at an art exhibition at the end of the semester. The course also focuses on works of art created by real artists to help students connect their work in class to real-world examples they can strive towards. With a similar week-by-week structure, we expose students to different ways to code creatively in our course, although the coding will be more advanced since students will have some computational skills coming into the course.

2.6.3 Takeaways

These courses were essential to study as they provided valuable directions for our course curriculum. The articles delineated key aspects of creative coding necessary for the success of our program. However, it is important to note that these courses were not designed for the same demographic as our target students; therefore, certain elements are not directly applicable, such as the learning objectives. Nonetheless, the insights gained from these articles significantly influenced the development of our creative coding course in multiple ways.

2.7 Summary

This section covered the concept of creative coding, an interdisciplinary field that merges programming with artistic expression, utilizing tools such as Processing and p5.js to create immersive digital artworks and interactive experiences. It also explored the pedagogical approach of active learning, which enhances student engagement and fosters innovative thinking by involving students in activities and discussions, proving particularly effective in creative coding environments. Additionally, the benefits of creative coding and active learning were illustrated through case studies, including primary school programs and university courses like

VA345 at Sabanci University, showcasing the adaptability of creative coding across educational levels and its positive impact on learning outcomes.

3. Methodology

The goal of this project was to formulate and present a creative coding course syllabus that could be used in a continuing education course for life sciences students. To reach this goal, we identified four research objectives.

1. Evaluate the effectiveness of different teaching strategies in the context of creative coding.
2. Identify and analyze the structures of a sample of existing creative coding courses.
3. Construct our Creative Coding Course Syllabus.
4. Measure the interest and effectiveness of our syllabus.

Within this chapter, we describe the methods used to achieve each research objective.

3.1 Objective 1: Evaluate the Effectiveness of Different Teaching Strategies in the Context of Creative Coding

An analysis of teaching strategies employed in academia was necessary to design a creative coding course syllabus and develop course materials. Active learning, a teaching approach gaining popularity in coding, was suggested as an area of focus to base the course on. To evaluate the teaching strategies that incorporated active learning in course instruction, we wanted to interview professors and students teaching and enrolled in such courses respectively. The interviews were intended to answer many fundamental questions about the course we designed: Where do students think is the balance between guided instruction and independent exploration? What specific approaches led to the most student engagement? What challenges did instructors experience in facilitating the course?

3.1.1 Semi-Structured Interviews with Course Instructors

Course instructors were interviewed to understand the effective ways to implement active learning strategies. To satisfy this data need, we conducted semi-structured interviews. The IQP group incorporated a mix of convenience and purposive sampling to define a pool of interviewees. Creative coding course instructors along with those implementing active learning at courses at Zurich University of Applied Sciences (ZHAW) were chosen as a target group for interviews. Those groups were meant to complement one another to understand the procedure of

the course syllabus design as well as focus on the specific examples of the course structure in the context of creative coding. This also helped to leverage the resources provided by ZHAW and ensure that cultural considerations of Swiss higher education were met. A total of 4 semi-structured 30-minute-long interviews were conducted with questions that can be found in [Appendix B](#).

3.1.2 Survey with Students About the Learning Strategies They Find Effective

Students are a key stakeholder group whose input was essential in shaping the course syllabus. Two surveys were planned to gather this data. In order to collect insights on the delivery of creative coding teaching materials, the first survey targeted students enrolled in the creative coding course. The "Creativity for the Arts" creative coding course taught at Zurich University of the Arts (ZHdK) with 8 students enrolled was a suitable course to collect student feedback. The second survey targeted students from various programming courses previously taught at ZHAW, aiming to collect insights on the teaching strategies and the course structure that students found effective. The survey was sent out to the students our sponsor was an instructor of in the past, thereby incorporating purposive sampling. After collecting the responses, they were de-identified and stored in the Google Drive folder accessible to group members only. We intended to draw from both technical and creative perspectives when considering two student groups.

3.2 Objective 2: Identify and Analyze the Structures of a Sample of Existing Creative Coding Courses

Prior to beginning the development of the creative coding course syllabus, it was essential to conduct interviews and meetings with experts in the field, as well as to gather existing creative coding course syllabi. The semi-structured interviews with creative coding professionals played a pivotal role in our process, as their insights helped to shape the most effective structure for the key components of our syllabus. Collecting existing creative coding course syllabi was also vital, as they provided a foundational reference for the construction of our own syllabus. The primary inspirations drawn from these syllabi included core course topics, resources, and overall structure.

3.2.1 Semi-Structured Interviews with Creative Coding Experts

The specific creative coding experts that were carefully selected were instructors who have relevant experience teaching this subject, our project sponsors, and other professionals in the field recommended by the aforementioned contacts. We conducted a total of nine semi-structured interviews, each lasting approximately 30 minutes, with creative coding experts from ZHAW, Zurich University of the Arts (ZHdK or Zürcher Hochschule der Künste), and Pratt Institute in New York, which offers a creative coding course developed by one of our sponsors. These interviews provided key insights that helped to guide the development of our course syllabus, particularly regarding the course audience, structure, project and assignment concepts, course objectives, available resources, and creative coding tools. The questions used in these interviews are detailed in [Appendix A](#) and [Appendix B](#).

We conducted semi-structured interviews with professors specializing in creative coding to gain insights into their pedagogical approaches to course development in this field and to explore how their academic backgrounds influenced their course structures. The selection of institutions for these interviews was based on our geographical location and the professional connections of our sponsors. The Zurich University of Applied Sciences (ZHAW) served as the primary site for all project work, while the Zurich University of the Arts (ZHdK), a local institution, offered its own creative coding course tailored for art students. Pratt Institute in New York was included in our study due to its creative coding course, which was originally developed by one of our sponsors, Dr. Tugçe Bilgin Sonay. Through our interviews with experts from ZHAW, we sought to better understand the coding backgrounds of its students in order to inform decisions regarding potential course prerequisites and appropriate levels for our syllabus.

The collected interview data was analyzed using a structured codebook. This codebook was organized around the key concepts we aimed to explore through the interviews, with each code representing a primary theme and accompanied by a detailed description of how the theme could be applied. The interview notes were meticulously reviewed, and whenever a relevant theme emerged, a corresponding comment was inserted alongside that section of text to highlight the identified theme. This method allowed for more efficient retrieval of specific topics during the syllabus development process. The contents of the codebook are detailed in [Appendix F](#).

3.2.2 Obtain and Analyze Creative Coding Course Syllabi

Creative coding course syllabi were acquired through the semi-structured interviews conducted with professors who had previously taught or were currently teaching creative coding courses, as outlined in [Section 3.2.1](#). We determined that it would be more effective to request syllabi during these interviews, rather than contacting the professors in advance without prior introduction.

The analysis of the collected syllabi was conducted using the same structured codebook described in [Section 3.2.1](#). After obtaining each syllabus, it was carefully reviewed, and notes were recorded in a separate document. These notes were subsequently coded using the same process outlined in [Section 3.2.1](#), with particular attention given to key themes such as course resources, objectives, projects, topics, and overall structure.

3.3 Objective 3: Construct our Creative Coding Course Syllabus

The most critical deliverable of this project was creating the syllabus itself. We used an iterative design process to develop each part of the syllabus, such as projects, lectures, and prerequisites. To ensure our syllabus stands out to ZHAW students, we also researched multiple possible exemplars to include as creative prompts, providing students with some familiarity when exploring their creativity.

3.3.1 Process for Making the Syllabus

Since the syllabus serves as the primary deliverable of the project, we ensured that the course itself is of high quality and well thought out; we created a design process to help build out the syllabus effectively. The step-by-step approach we used is as follows:

1. Organize research into a course framework
2. Understand the audience who would take the course
3. Identify key goals of the syllabus
4. Establish topics and create course materials

We organized the research done in both the background research done for the project and the additional research done in objectives 1 and 2 into an overall syllabus framework. Here, we detailed essential aspects to focus on within the course structure and the direction we took with our course. We organized our research this way because it ensured that the course we created

would be based on key findings of other courses, resulting in a more logical and high-quality syllabus. We identified the course goals to establish a clear direction for our syllabus. Here, we defined the main focus of the course and the outcomes we wanted students to achieve in the course, such as new skills or perspectives. Another critical step was understanding the students themselves. We understood that creative coding could be for anyone. However, we wanted to specifically reach STEM students who don't have much creative experience. That is why we interviewed professors who have taught similar courses to better understand the students who could take this course. With the foundation laid out, we identified the topics to focus on in the syllabus. We created the content that would accompany the topics, such as the teaching strategies used and the materials the students would require, including the assignments and projects they would complete throughout the course, and organized it all under a weekly calendar.

3.3.2 Develop Familiar Topics to Serve as Exemplars for Students

When creating the course topics, we realized students may be unfamiliar with the creative process. We wanted to overcome that unfamiliarity by introducing relatable and interesting creative topics for STEM students. We accomplished this by focusing on identifying creative prompts for projects that are related to life science-related topics. Through interviews with professors and discussions with our sponsors, we identified possible topics to make great projects for the targeted students. This also allowed us to create our creative coding pieces to use as exemplars to drive interest in the course and provide something physical to connect with unsure students.

3.4 Objective 4: Evaluate the Interest and Effectiveness of the Syllabus

The primary purpose of this objective was to gather feedback about the syllabus. This was done to ensure the finished syllabus's quality and gauge how interesting and effective it would be if it were to be taught at ZHAW. We accomplished this by surveying students about their interest in creative coding and the possibility of taking a course about creative coding. We also interviewed professors to get their feedback on the syllabus.

3.4.1 Interest

We wanted to prove that the course we created would appeal to students. So, the only way to figure that out was to ask them directly. Here, we sent a survey, detailed in [Appendix C](#), describing the course we created and whether or not the students would take the course based on the description, including some example artwork to represent the course better. We also asked for their feedback on the course so we could implement it into the course as needed.

3.4.2 Effectiveness

We also wanted to prove the quality of the overall syllabus and whether or not the content could be taught effectively based on the structure of the syllabus. In this case, we interviewed professors, including our sponsors, about how practical the course would be at teaching creative coding if it were to be taught by the professors that day. Using the outlined questions in [Appendix D](#), we planned to interview at least three professors. We received feedback on the quality of the syllabus and whether the teaching strategies we implemented were a good fit for the course and the students at ZHAW.

3.5 Summary

The project developed a creative coding course syllabus by analyzing existing courses, conducting interviews with experts, and reviewing syllabi. Active learning strategies were assessed through interviews with instructors and students, focusing on engagement and course challenges. The syllabus was built using an iterative process, incorporating feedback and exemplar projects relevant to STEM students. Surveys and interviews were conducted to evaluate student interest and the overall effectiveness of the course design.

4. Results and Discussion

This chapter presents the results gathered during the Project. Four key topics are discussed.

1. Teaching Strategies Results: Student collaboration improves learning, pre-recorded lectures improve focus, and quantity-based tasks inspire creativity.
2. Course Goals, Audience, and Content Results:
 - a. Course Goals Results: Develop an understanding of the creative process, enable students to feel competent in Python syntax, basic programming structures, and the use of libraries in coding applications, explain the core concepts of creative coding and equip students with the skills to implement basic machine learning algorithms.
 - b. Tools, Resources, and Audience Results: Python is the main programming language for the course. Turtle, Pygame, Numpy, Matplotlib, and ml4a will be the tools used during the course. The course topics will be Turtle, Pygame, Game of Life, and Machine Learning.
3. Course Syllabus
 - a. Course Structure: Designed an 8-week program for students with a basic understanding of programming and Python. Through a flipped classroom environment, students learn to apply the creative process to programming to create visual art, all of which culminate in a course project to assess their skills and creativity.
 - b. Week-by-week schedule: Created a schedule detailing all the information the students need to accomplish each week. This includes pre-lecture materials and reading, lecture plans, and assignments surrounding the topic of that week.
4. Interest and Effectiveness of the Course Results
 - a. Effectiveness Results: Received feedback from instructors about the syllabus. Some feedback directly changed our final syllabus, such as the structure and wording of the learning objectives. Some other feedback was suggestions for different directions to take the course, such as other possible topics the course could have featured, for example, fractals with Python.

Each of these sections incorporates findings based on data collected throughout the course of our research.

4.1 Teaching Strategies Results

To evaluate different teaching strategies and better understand how to structure the syllabus, five semi-structured interviews were conducted with instructors of creative coding courses. All with different backgrounds and course audiences, the instructors were found to utilize bits of active learning teaching strategies, including flipped classrooms, pre-recorded lectures, and project-based, and collaborative assignments.

4.1.1 Student Collaboration Improves Learning

A collaborative environment in creative coding classrooms has proven to foster students' learning while inspiring creativity and curiosity. This was a common theme reported by every creative coding instructor interviewed. One professor, teaching a continuing education course with a Certificate of Advanced Studies (CAS), introduced an assignment where students translated their computer-generated designs onto paper using pen plotters. When we walked into the classroom for the scheduled interview, students were already working in pairs, guiding the machine to print their designs.

Yet another instance of student collaboration that two professors related was around “debugging” the code during the class. Unlike in other fields, programming is especially known for the variety of ways to solve one coding problem. Therefore, students have preferred consulting with their peers on coding tasks instead of following the examples provided by the instructor. Open-sourcing the code on the final projects that students completed served as an inspiration for other students to solve problems in different ways, claimed the instructor teaching creative coding to students with no previous programming experience. Based on this information, flipped classrooms, where students collaborate on assignments during class, are incorporated into the lectures to enhance creativity and peer learning.

4.1.2 Effective Instruction Techniques

A second significant finding that confirms the claims of active learning research was made during the interview process ([Prince, 2004](#)). Specifically, each of the professors interviewed reported diminished student interest and attention with the increased lecturing time. This is attributed to a general loss of interest when the classes are occupied by lecturing, but also

to the specific nature of learning to program ([Bunce, Flens, & Neiles, 2010](#)). That is, one of the professors explained that in order to learn and retain a particular programming concept, a student must code on their own instead of looking at the program written by the instructor.

To make the classes more engaging, one of the instructors we interviewed recorded short videos that the students had to watch before the lecture. This way, he stated, the students came ready to implement the concepts learned before the class. Another instructor found that releasing the assignment before the lectures was an effective way of shortening the lecture time to no more than 15 minutes and spending the rest of the class providing individual feedback to students struggling with a particular assignment. During the interview with a lecturer of a creative coding course taught to first-year undergraduate courses, she highlighted the importance of some coding knowledge to start operating this medium creatively. To address this, she created a nine-to-five crash course a week before the start of the first class. This, she explained, was useful for students to learn the basics of code and gave them a week to catch up on the concepts they were unfamiliar with before the first class.

Students will be assigned comprehensive readings that introduce key topics before class, allowing them to engage with the material at their own pace and arrive prepared with questions. This approach aims to facilitate active participation and concept application during class.

4.1.3 Creativity-Specific Teaching Strategies

Creative coding-specific teaching strategies were discussed with course instructors. One specific finding, which is not documented in the academic literature to our knowledge, was made during the interview with the professor teaching a CAS creative coding course. Specifically, he stated that students produce creative artwork when the assignments are evaluated based on the quantity of works instead of the quality of a single piece. This idea was borrowed from a story in *Art & Fear* ([Bayles & Orland, 1993](#)). In the story, a ceramics teacher divides the class into two groups: one group is graded solely on the quantity of work they produce, measured by the total weight of their pottery. In contrast, the other group is graded on the quality of a single piece. Counterintuitively, the authors of the book state the most professional and creative pieces tend to emerge in the group focused on quantity.

Learning the creative process is about pushing the boundaries of what one can imagine. By focusing on quantity, students are compelled to produce work, and through this process of

continuous creation, new ideas naturally emerge, leading to greater creativity and development. Introducing a quantitative metric into the evaluation was reported to be one of the most effective ways of fostering creativity in students. Therefore, iterative after-class assignments were introduced in the syllabus to encourage creativity by focusing on quantity and reducing the pressure to achieve perfection on the first try.

4.2 Course Goals, Audience, and Content Results

4.2.1 Course Goals Results

The main goal of this course is to give students a solid understanding of both Python programming and the creative process, helping them explore and innovate at the intersection of art and technology. The course is meant to help students become comfortable with Python syntax, basic programming structures, and using libraries in coding applications, which will be the base for their creative projects. By improving these skills, students will be able to use Python to create simple visual art projects and animations, turning their ideas into tangible digital expressions.

A key learning outcome of the course is for students to discuss the creative process and how it can be integrated into problem-solving. A creative coding professor from the Zurich University of the Arts (ZHdK) highlighted the importance of iterative experimentation. While a rigorous iteration model, like producing 100 versions of a project in one month, may not be feasible in an 8-week course, the idea behind it remains central. Students will be pushed to step outside their comfort zones and improve their work through multiple iterations, learning to accept the trial-and-error nature of creativity in coding.

In addition to the creative process, students will explain the core concepts of creative coding, particularly how programming can be applied to artistic endeavors. Professors we spoke to agreed that students should not feel pressured to become full-fledged artists. Instead, they will learn to break down the components of visual art projects, such as loops, conditionals, and functions, to see how programming elements combine to form cohesive, creative outputs. This will help students gain insight into the relationship between code and art, guiding them as they develop their own artistic style.

The integration of machine learning plays a pivotal role in expanding students' creative toolkit. Professors from various institutions underscored the importance of machine learning in

modern creative coding. Even a basic understanding of algorithms such as decision trees or neural networks will allow students to apply basic machine learning algorithms to generate creative outputs. This knowledge will also help them analyze the behavior of cellular automata and machine learning models in creating complex patterns and visuals, broadening their capacity for dynamic and innovative work.

Ultimately, the course aims to help students design and develop a final project that demonstrates a personal creative coding approach, merging their technical skills in Python with their individual artistic vision. By the end of the course, they will have not only improved their coding knowledge but also gained the confidence to explore and create within a new, hybrid domain of art and technology.

4.2.2 Tools, Resources, and Audience Results

Using Python as the primary programming language for our course syllabus was determined to be the most effective choice. After conducting interviews with nine experts in creative coding, it became evident that JavaScript, specifically for creative coding purposes, would have been the optimal choice for the course. Each professor we consulted recommended various JavaScript libraries and tools, such as p5.js and ml5.js, as ideal for creative coding. However, while JavaScript may have been the preferred language for creative coding, experts from the Zurich University of Applied Sciences (ZHAW) suggested that Python would be more suitable for the student demographic. According to these interviewees, Python was one of the most widely used, versatile, and accessible programming languages available. Furthermore, many students at ZHAW already had foundational knowledge of Python, with a researcher noting that Master's students typically completed at least one Python course by the end of their first semester. Although Python may not have been the most optimal language for creative coding, it still offered creative coding capabilities and was ultimately deemed the best choice for the intended audience. This decision subsequently guided our determination of the course prerequisites. It was concluded that a foundational knowledge of coding and Python would be required for enrollment, with no additional prerequisites imposed in order to maintain accessibility for a broader audience.

Creative coding tools that could support students with foundational coding experience in exploring this concept were also identified through our expert interviews. Once Python was

selected as the primary programming language for our course syllabus, the next step was determining which tools would best facilitate students' understanding of creative coding. Python offered a broad range of coding libraries and was compatible with various integrated development environments (IDEs). A creative coding instructor from the Pratt Institute in New York introduced us to several libraries used in her course, both through an interview and her course syllabus, which she shared with us. For instance, she recommended Turtle and Pygame, two libraries that aid in visualization and game development, respectively. There is some divergence between her course and ours, as her course is designed as an introductory programming course spanning an entire semester. In contrast, our course is designed for students who have already acquired foundational programming knowledge, and it spans a duration of only 8 weeks.

Several interviewees also emphasized that students should be allowed to choose their preferred IDE rather than mandating a specific one, to ensure they could work with the environment they were most comfortable with. Ultimately, the creative coding tools we chose are Turtle ([Python Software Foundation, n.d.](#)), Pygame ([Pygame Community, n.d.](#)), NumPy ([NumPy Developers, n.d.](#)), and Matplotlib ([Matplotlib Development Team, n.d.](#)). Turtle and Pygame both serve as effective reintroductions to coding and introductions to creative coding in Python, NumPy supports computational tasks, and Matplotlib assists in teaching animated visualizations. The inclusion of NumPy and Matplotlib was inspired by an interview with a ZHAW researcher who had expressed a strong interest in cellular automata.

Turtle and Pygame

As outlined in our decision regarding creative coding tools, Turtle and Pygame were selected to reintroduce fundamental coding concepts and to support the initial stages of students' creative coding journeys. These two libraries proved instrumental in developing creative coding skills in an introductory course at Pratt Institute, which led us to include them as key topics for introducing creative coding in our course. The course began with an introduction to Turtle, which offered accessible functions for generating visuals while allowing for the creation of complex outputs. Once students have a solid grasp of Turtle, Pygame is introduced. Although slightly more challenging than Turtle, Pygame remains approachable and serves as a valuable tool for students to explore the fundamentals of game development.

Game of Life

Conway's Game of Life was highlighted during an interview with a research scientist at ZHAW ([Conway, 1970](#)). The Game of Life is a cellular automaton where cells on a grid evolve based on simple rules, simulating population dynamics. The biological patterns, such as cellular processes and the study of systems behavior, make this topic resonate with the Institute of Computational Life Sciences at ZHAW, our project sponsors. The Game of Life has also become a creative medium where emergent patterns can be explored (Figures 1 and 2). This combination of interest from students and artistic potential created a compelling case for including Conway's Game of Life as one of our course topics.



Figure 1. Duel, generated with Game of Life ([Kjetil Golid, circa 2020](#))

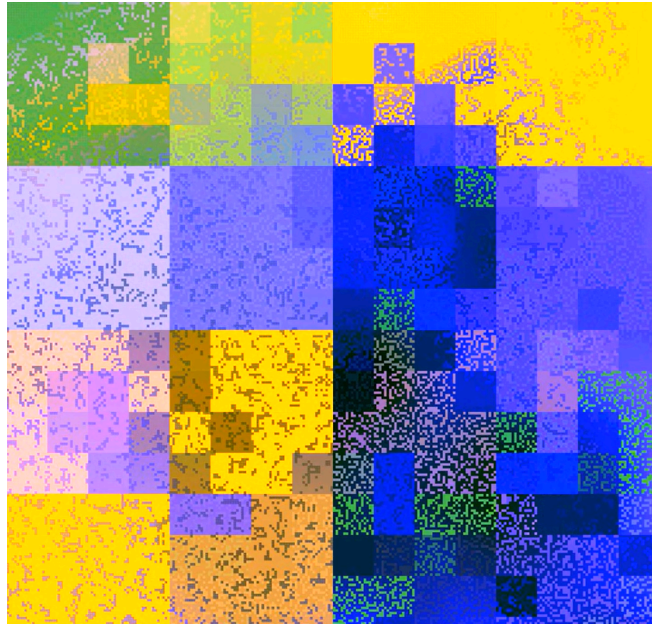


Figure 2. Ccnwwys, generated with Game of Life ([Gamboa Maon, 2020](#))

The exemplars to advertise the course and showcase what students will be working on were produced based on Conway's Game of Life. The creative and visual outputs are generated by changing the rules of the game. In the course exemplar, we implemented Multiple Neighborhood Cellular Automata (MNCA), a proposal of a set of algorithms introduced by Slackermanz ([Slackermanz, n.d.](#)). In our implementation, two distinct neighborhoods—von Neumann and Moore—are defined to determine cell behavior. Cells evolve based on average activity in two circular neighborhoods with radii of 2 and 5 units. The smaller neighborhood triggers changes if its average activity is between 0.22 and 0.34, increasing the cell's value by 0.12, or if between 0.32 and 0.4, adding 0.08. If these conditions aren't met, the cell's value is decreased by 0.02. The larger neighborhood also impacts the cell, reducing its value by 0.14 when the average activity is between 0.12 and 0.42. These conditions allow for complex, evolving patterns based on local interactions, showing how altering neighborhood definitions and thresholds can lead to diverse, dynamic outputs.

Students are introduced to the Game of Life by going through the Google Colab notebook created by the IQP group ([Appendix E: Google Colab, 2024](#)). It first introduces the basic rules of the Game of Life: if a cell is alive, it survives if it has 2 or 3 neighbors, dies from overpopulation if it has more, and from isolation if fewer. The notebook then dives into more complex rules and

optimization techniques for rule implementation and video rendering. Along each step, students are asked to complete the critical bits of the code to ensure their solid understanding of the code. As a result, a simulation is rendered similar to that in Figure 3 where cells self-organize into larger conglomerates and start moving through the board with each iteration. Students are then encouraged to develop their own rules and produce a creative output as on the described assignment.

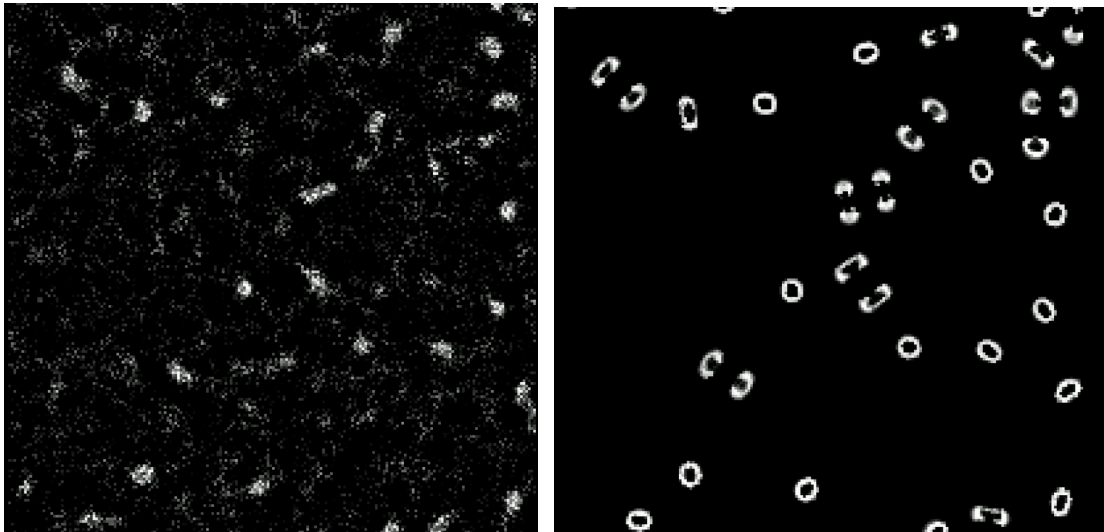


Figure 3. Generated and visualized Game of Life with MNCA rules. (a) the first frame of the simulation, (b) the last frame of the simulation

Machine learning (ML)

Course instructors responded to the rise in popularity of Artificial intelligence (AI) by introducing machine learning as a concept in creative coding courses. Underscoring this point, 4 out of 5 course syllabi we reviewed have machine learning as one of the focus areas of the course. On top of the relevance to computational life sciences, introducing machine learning serves as a unique tool for generative art.

A wide range of creative coding ML tools and libraries were recommended by creative coding instructors. These ranged from sound generation libraries like Sonic Pi ([Aaron, n.d.](#)) and Touchdesigner ([Derivative, n.d.](#)), to image generation tools such as ml5.js ([Mok, n.d.](#)) and OpenFrameworks ([Lieberman & Reas, n.d.](#)). After considering different options, a choice was made in favor of ml4a ([Kogan, n.d.](#)), the one famous Python-based creative coding tool. The introduction to ML in the course is guided through the published ml4a electronic textbook

([Kogan, n.d.](#)) and a lecture by the course instructor. Students spend the next class implementing an algorithm with a choice between “[Glow:Reversible Generative Models](#)” ([Kogan, 2020](#)), “[DeepDream:hallucinate with a neural network](#)” ([Kogan, 2021](#)), or any other algorithm in the ml4a library.

Final Course Project

A course-long project is a central element in achieving the learning outcomes of the creative coding courses reviewed. One professor emphasized that allowing students to explore their own ideas is where the creative process truly begins to flourish. Throughout the course, students work on a project that reflects their personal interests and intersects with creative coding. Midway through the course, students submit an abstract that details their project scope, receiving feedback from the instructor to refine their direction. This mid-course feedback has been noted by instructors to significantly enhance the quality of the final work. Students dedicate Week 7 in-class time to collaborative project work, allowing them to refine their ideas and receive peer feedback. On the last day of the course, students present their completed projects in 5-minute presentations, showcasing their creative journey and the outcomes of their efforts.

4.3 The Course Syllabus

The creative coding course syllabus was designed as an eight-week program for students with foundational Python skills to explore programming through creative applications (Table 1 and Table 2). Covering topics like digital drawing, game development, cellular automata, and machine learning, the course integrates active learning techniques throughout its structure. Flipped classroom techniques are implemented by introducing each new topic through assigned readings or pre-recorded lectures before class, allowing students to come prepared for hands-on activities during in-class sessions. Assignments, such as the Week 3 (Figure 5) task where students devise new rules for Conway's Game of Life, are structured to challenge students to push their creativity beyond conventional boundaries. Collaboration is incorporated by organizing small group work during project sessions and encouraging students to work together during coding exercises, fostering a learning environment where peer interaction and teamwork are prioritized.

Table 1. The Creative Coding Course Syllabus ([Google Docs, 2024](#))

Course Description	Unleash your creativity and transform code into art in this eight-week course about creative coding. Students with basic programming experience who want to explore artistic expression or enhance their programming skills, specifically with Python, are welcome to join. Students will learn about exciting topics such as Conway's Game of Life, cellular automaton simulating intricate patterns, and Machine Learning, where algorithms learn to create art. Through a flipped classroom environment, students will engage with core concepts on their own time with pre-lecture videos, allowing students hands-on sessions during lectures that encourage exploration of creativity. Students will create multiple visual art projects throughout the course, culminating in a final project where they can choose what they want to make based on their interests. At the end of the course, students will leave with deepened programming skills and hands-on experience with creativity.
Workload	3 ECTS (84 student working hours: eight 3-hour lessons for 24 hours of in-class work, and 60 hours of outside-of-class work)
Entry Requirements	This module requires fundamental programming knowledge in Python (e.g., conditionals, loops, lists/arrays).
Learning Outcomes and Competences	After completing the module, students will be able to: - feel competent in Python syntax, basic programming structures, and the use of libraries in coding applications - explain the core concepts of creative coding, including the principles behind science-related programming topics and basic machine learning algorithms - discuss the creative process and how it can be integrated into problem-solving - use Python programming to create simple visual art projects and animations - apply basic machine learning algorithms to generate creative outputs - break down the components of visual art projects to identify how different programming elements (e.g., loops, conditionals, functions) contribute to the overall outcome - analyze the behavior of cellular automata and machine learning models in creating complex patterns and visuals - design and develop a final project that demonstrates a personal creative coding approach, combining programming skills with artistic expression

	The objective of this module is to provide students with an environment that merges their programming skills with the creative process. By the end of the course, they will be equipped to apply what they have learned to future projects, both inside and outside the classroom.
Module contents	The module covers the following topics: - Introduction to Creative Coding with Digital Drawing (using Turtle programming library) - Introduction to Basic Game Development (using Pygame programming Library) - Exploration at the intersection of Science and Creative Coding - Advanced Creative Coding with Machine Learning Algorithms (using ml4a) - Design of a Course Project
Course Project	This project focuses on the application of creative coding concepts learned throughout the course. Emphasizing the development process over the final product, students will engage in experimentation, iterative problem-solving, and exploration of techniques. The primary learning outcome is to deepen students' understanding of creative coding concepts while also enhancing their critical thinking and technical skills.
Teaching/learning methods	The module will consist of pre-recorded lectures, collaborative assignments, in-class exercises, and a final project. The course is based around a “flipped classroom” that incorporates an active learning pedagogy.
Assessment of learning outcome	Based on completion. Pass/fail
Format	8 weeks

Table 2. The Course Schedule ([Google Docs, 2024](#))

Week	Lecture Title + Plan
Week 1	Introduction to Creative Coding with Digital Drawing (no pre-recorded lecture prior to class because this is the first lesson) 1) The instructor begins class by familiarizing students with the concept of creative coding and its applications. 2) The basics of the Python Turtle library are taught by the instructor, focusing on

	simple commands and how to manipulate the turtle to create graphics. 3) Students are encouraged to explore and create more complex designs using loops and Turtle commands. Students group up and discuss how loops can be implemented alongside Turtle and the potential outputs that can be produced. 4) Students are given an opportunity to apply what they have learned by creating a simple line-drawing landscape image that must be submitted online by the night before the Week 2 lesson. The instructor will review each assignment and return feedback on the code provided by each student before the Week 3 lesson. An example of a project would be a two-story house with a tree next to it. Helpful resources: Collection of all Turtle methods (Turtle, n.d.) Simple shape creation examples (GeeksforGeeks, 2017)
Week 2	Introduction to Basic Game Development Pre-recorded lecture concepts: 1) The instructor introduces the Pygame library and basic concepts of 2D game development. 2) Students are taught by the instructor how to set up Pygame and work with basic functions for window creation and event handling. During class: 1) The instructor presents the Course Project. 2) The instructor introduces dynamic interaction by moving objects using the game loop and event handling. 3) The assignment described below is introduced. Students are encouraged to form teams of 2-3 people to discuss ideas for the output. The students will work together on the assignment and one student will submit it. Students are encouraged to discuss their preferred way of working on the assignment outside of class (meeting up in person, working remotely, etc.). Assignment: 1) Students apply learned skills by developing a small interactive game that must be submitted online the night before the Week 3 lesson. The game must include at least 3 user interaction components that generate different on-screen results. The instructor will review each assignment and return feedback on the code provided by each student before the Week 4 lesson. Helpful resources: “Introduction to Pygame” (Pygame Community, n.d.) “Pygame Basics” (Sweigart, n.d.) Sources for event handling (GeeksforGeeks, 2021)
Week 3	Exploring the Intersection of Science and Creative Coding: Introduction

	Before class: The students are assigned the reading of “Mathematical Games” (Gardner, 1970), a 5-page paper that introduced Conway’s Game of Life in the Scientific American magazine. During Class: 1) The class starts with an overview of the coding functions to be used; the students create Conway’s Game of Life by filling the gaps in the code of the Google Colab Notebook (Google Colab, 2024; Appendix E). 2) By modifying the created code, the students create a program that defines an initial condition (which cells are dead/alive) and tests various evolving patterns. Students then share the results of simulations with peers. Assignment: 1) <i>Course Project</i>: By the night before the Week 4 lesson, students submit 3 ideas, each a paragraph long, that they have for the Course Project. Students are encouraged to think and explain how the skills learned in the class will be applied during the Course Project. The instructor will review all of the Course Project ideas and return feedback before the Week 5 lesson. 2) Students are tasked to test 3 initial conditions with patterns of their choice, producing a creative output. Students are tasked to think about the following questions to prompt ideas: What happens if you introduce 2 patterns in one game? What happens if you introduce 2 self-destructive/reinforcing “alive” states? This assignment must be submitted the night before the Week 4 lesson. The instructor will review each assignment and return feedback on the code provided by each student before the Week 5 lesson.
Week 4	Exploring the Intersection of Science and Creative Coding: Continued Before class: Students are assigned the reading of “Understanding Multiple Neighborhood Cellular Automata” (Slackermanz, 2021) and “The Game of Life - Emergence in Generative Art” (Bailey, 2020) During Class: 1) The class starts with an overview of the new coding functions to be used in the example; the students create Multiple Neighborhood Cellular Automata (MNCA) cellular automata by filling the gaps in the code of the Google Colab Notebook (Google Colab, 2024). 2) Students then modify the code by experimenting with different region parameters and scales. Students share and discuss the outcomes of their experimentation with peers. 3) In the same framework of MNCA, students are tasked to implement 3 creative visual outputs of games with completely different regions and rule sets. Students are encouraged to define different MNCA regions. They may also consider defining the last frame of the previous run as the starting frame for a new one.
Week 5	Machine Learning: Introduction

	Before class: Students are assigned to watch Kogan's lecture (Kogan, 2018) about neural networks by Gene Cogan, an artist teaching ML. The readings of Chapters 1, 2, 3, and 4 on Neural Networks from the <i>Machine Learning for Artists website</i> (Kogan, n.d.) are to be used as a reference. During class: 1) The class starts with an introduction to machine learning, tensor operations, model types, and ways of training them. 2) An introductory number recognition algorithm with X hidden layers is implemented in class together with the course instructor. The neural network is trained on the MNIST dataset (LeCun, Cortes, & Burges, n.d.) and training methods like gradient descent are explained along the way. Assignment: 1) <i>Course Project</i>: With the received feedback from the instructor, students submit the project proposal, expanding on the idea they chose. They briefly describe in pseudocode the strategy for implementing their idea. The project proposal must be submitted the night before the Week 6 lesson. The instructor will review each proposal and return feedback to each student before the Week 7 lesson.
Week 6	Machine Learning: Continued During class: 1) The instructor chooses an algorithm from the ml4a library like adapted Deepdream (Kogan, 2021), GLOW (Kogan, 2020), or any other to implement during class time. The students read about it before the class and implement it with the instructor during the class. 2) Students break into groups to work on the assignment described below. This will not be a collaborative assignment outside of class, but students are encouraged to use class time to discuss what variations of the algorithm they are planning to use and why. Assignment: The students are to produce 3 creative images by changing the parameters of the model and playing with the input. For example, in the case of Deepdream, the students will run different images as input and implement various masks as parameters. This assignment must be submitted the night before the Week 7 lesson. The instructor will review each assignment and return feedback on the code provided by each student before the Week 8 lesson.
Week 7	Course Project: Collaboration and Peer Feedback 1) The instructor divides students into groups based on their project topics. The students talk about the challenges of their project and share ideas. 2) The students get feedback from the instructor on the areas of improvement and ways of resolving particular challenges.

	3) The students work on the project.
Week 8	Course Project: Presentation 1) The students present their course projects, discussing the creative outcome, technical challenges faced, and solutions implemented.

4.4 Interest and Effectiveness of the Syllabus

The final objective was to implement mechanisms to measure student interest and assess the effectiveness of the syllabus we created.

4.4.1 Interest

A student survey was planned to assess interest in the course. However, due to limited access to the student body early in the project (as the academic year began mid-September) and the syllabus being finalized late in the WPI term, the decision was made not to conduct the survey. A survey was developed with the planned target audience of students enrolled in the Master's program at Zurich University of Applied Sciences, specifically those in the Institute of Computational Life Sciences (ICLS) ([Appendix C](#)).

4.4.2 Effectiveness

A total of three interviews were conducted with course instructors and researchers from ICLS regarding the effectiveness of the Creative Coding course syllabus. Their insights led to significant modifications to the course syllabus.

A course instructor at ICLS suggested generalizing course themes to make it easier for instructors to adjust content and for students to understand the main ideas, even if they aren't familiar with specific topics. For example, a two-week module on Conway's Game of Life is presented as a theme in "Exploring the Intersection of Science and Creative Coding," reflecting the thought process during the syllabus design without losing clarity.

A number of suggestions to improve the quality of the course have not been incorporated into the course syllabus but are valuable to acknowledge:

Fractals with Turtle. Coding and visualization of various fractals using the Turtle library in Week 1 was recommended as a beneficial addition to the curriculum. This approach aligns well with the emphasis of the course on creative representations of scientific concepts.

Game of Life with Pygame. During one of the syllabus reviews, it was suggested that incorporating Conway's Game of Life into Pygame ([Pygame Community, n.d.](#)) would provide a smooth transition between the second and third weeks as the material becomes more challenging while the platform stays the same. A future instructor of the course can also make use of the existing GitHub library ([Eric-the-Geric, n.d.](#)) shared by one of the interviewees that integrates Conway's Game of Life with Pygame.

Various **self-organizing systems** were discussed as potential course additions. These include flocking algorithms and oscillators, which offer dynamic behaviors and foster creative exploration as alternatives to Conway's Game of Life. Self-organizing systems involve structures emerging from local interactions without external control, and these approaches are well-researched and worth exploring. References we explored for flocking algorithms include Boids ([Wong, 2008](#)), p5.js sketches by Codeheir ([Codeheir, n.d.](#)), and more detailed descriptions by Codeheir and Conrad ([Codeheir, 2021; Conrad, n.d.](#)). For oscillators, see Complexity Explorables ([Brockmann, n.d.](#)).

In addition to the resources mentioned above, a lot of information on creative coding has been gathered during the interview process. This included course topics, readings about those topics, course websites, and creative coding libraries. These resources have been compiled into a Course Resources spreadsheet where they are organized in the separate tabs ([WPI Creative Coding Team, 2024](#)). It is intended to serve as an additional reading and course project inspiration resource for students.

5. Conclusions and Recommendations

This section presents the key conclusions and recommendations derived from our research for developing our creative coding course syllabus. Our results are based on insights gathered through interviews with instructors and an analysis of various course materials. These conclusions highlight the importance of fostering creativity, collaboration, and technical skill development within creative coding courses. Following this, we outline specific recommendations for instructors who may utilize our syllabus.

5.1 Conclusions

Through our research we obtained valuable insights into effective strategies for teaching creative coding, shaping a syllabus that balances technical and creative learning. Through interviews with creative coding instructors, several key themes emerged that helped guide teaching method construction and course design.

Firstly, **collaboration** was found to be a critical component in fostering creativity and enhancing problem-solving skills in the classroom. Students working together, particularly in coding tasks, were more engaged and generated more innovative solutions. This suggests that encouraging peer interaction should be a core strategy in creative coding education.

Secondly, the use of **active learning** techniques, such as flipped classrooms and shortened lectures, was shown to sustain student engagement and improve understanding. The findings highlight the importance of hands-on coding and personalized feedback, which are essential for mastering programming concepts and applying them creatively.

Additionally, a focus on **quantity over quality** in creative assignments, though counterintuitive, was found to promote more imaginative and varied student output. By encouraging students to produce multiple iterations, rather than perfecting a single piece, the creative process becomes more fluid, resulting in higher levels of innovation.

The integration of **machine learning** into creative coding courses offers students new tools to explore complex, dynamic projects. Even basic machine learning knowledge can significantly expand their creative capabilities, allowing for the generation of more sophisticated and interactive works.

Although **Python** may not be the traditional choice for creative coding, it proved to be an accessible and versatile language for the intended student audience of our Sponsor. With its rich set of libraries like Turtle, Pygame, and NumPy, Python successfully supported both the technical and creative aspects of the course, making it a suitable choice for our syllabus.

Aligning with our findings, the **eight-week course** introduces creative coding to students with basic Python skills. It covers topics like digital drawing with Turtle, game development with Pygame, Conway's Game of Life, and creative coding with machine learning algorithms. Using a flipped classroom model, students review pre-recorded lectures and engage in hands-on coding during class, fostering interactive learning. The course emphasizes both creativity and technical development, with students completing multiple visual art projects to apply their knowledge in innovative ways. This approach ensures students gain both the technical and creative skills needed to explore the intersection of art and code.

In summary, the findings of this study point to the importance of a balanced approach that fosters creativity through collaboration, active learning, and iterative design, while also equipping students with the necessary technical tools to excel in creative coding.

5.2 Recommendations

This section highlights recommendations based on our experience and research conducted during this project, specifically recommendations for lecturers who use our course syllabus.

5.2.1 Recommendations for Lecturers Who Use Our Course Syllabus

Understanding that we will not teach the syllabus we created, we include some recommendations for adjustments that can be made for lecturers interested in teaching the course.

Programming Languages and Programming Applications

We chose Python for the creative coding course to best align with the programming knowledge of many STEM students at the ZHAW. Most students at ZHAW take a Python course centered around data visualization in their first year and a Machine Learning course. Python was the preferred choice when designing this course, as it allowed us to focus more on creativity.

However, we recommend that lecturers choose to teach any programming language they feel best fits the course and students they teach. Creative coding does not require a specific language as long as the language supports visualization. For example, JavaScript is a common language used for other creative coding courses.

This is also true for programming applications. For our course, we mostly used Google Collab for creating and sharing code. However, any programming application would work, as preference over usability mainly dictates this. We recommend Google Collab as a versatile choice; however, we encourage lecturers to use the application they are most familiar with regardless.

Course Topics

For our course, we chose specifically to focus on Conway's Game of Life and Machine Learning through recommendations from our sponsors and other interviews we conducted. Part of our methodology was to create projects where STEM students can find some familiarity and relate what they make in creative coding back to their primary fields of study. Introducing topics to the course that are discussed in [Section 4.4.2](#) will ensure that the main theme is preserved while offering different topics.

5.2.2 Recommendations for Future Work on the Project

Gauging student interest is a critical next step before offering the course. Conducting the survey previously developed ([Appendix C](#)) will provide a good understanding of the audience. This insight will support aligning the course with student interests and needs while also providing a basis for offering the course.

Additionally, switching the mode of delivery of the course to “online” could also be considered. This would expand accessibility, allowing a broader audience to participate and potentially increase enrollment. An online format would also provide flexibility for students, making the course more attractive to individuals balancing other commitments. However, this shift requires careful adjustment of teaching strategies to maintain engagement and collaboration in the virtual environment.

References

- Amiri, F. (2011). Programming as Design: The Role of Programming in Interactive Media Curriculum in Art and Design. *International Journal of Art & Design Education*, 30(2), 200-210. <https://doi.org/10.1111/j.1476-8070.2011.01680.x>
- Anderson, E. F., Peters, C. E., Halloran, J., Every, P., Shuttleworth, J., Liarokapis, F., Lane, R., & Richards, M. (2012). In at the Deep End: An Activity-Led Introduction to First Year Creative Computing. *Computer Graphics Forum*, 31(6), 1852-1866. <https://doi.org/10.1111/j.1467-8659.2012.03066.x>
- Artut, S. (2017). Incorporation of computational creativity in arts education: Creative coding as an art course. *SHS Web of Conferences*, 37, 1028–. <https://doi.org/10.1051/shsconf/20173701028>
- Bailey, J. (2020, July 12). The Game of Life - Emergence in generative art. Artnome. <https://www.artnome.com/news/2020/7/12/the-game-of-life-emergence-in-generative-art>
- Bohrer, R. (2023). Centering humans in the programming languages classroom: Building a text For the next generation. Worcester Polytechnic Institute. <https://users.wpi.edu/~rbohrer/pub/splashe23.pdf>
- Brockmann, D. (n.d.). *Coupled oscillators*. Complexity Explorables. <https://www.complexity-explorables.org/topics/coupled-oscillators/>
- Bunce, D. M., Flens, E. A., & Neiles, K. Y. (2010). How long can students pay attention in class? A study of student attention decline using clickers. *Journal of Chemical Education*, 87(12), 1438-1443. <https://pubs.acs.org/doi/abs/10.1021/ed100409p>
- Caceffo, R., Gama, G., & Azevedo, R. (2018). Exploring Active Learning Approaches to Computer Science Classes. *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, 922-927. <https://doi.org/10.1145/3159450.3159585>
- Cardona-Rivera, R. E., Sullivan, A., & Young, R. M. (2019). *Using Ink and Interactive Fiction to Teach Interactive Design* (Vol. 11869, pp. 68–72). Springer International Publishing AG. https://doi.org/10.1007/978-3-030-33894-7_8

Chapter 2 — *Pygame Basics*. (n.d.). Retrieved October 2, 2024, from

<https://inventwithpython.com/pygame/chapter2.html>

Chibalashvili, A., Savchuk, I., Olianina, S., Shalinskyi, I., & Korenyuk, Y. (2023). Creative Coding as a Modern Art Tool. *BRAIN. Broad Research in Artificial Intelligence and Neuroscience*, 14(2), 115–127. <https://doi.org/10.18662/brain/14.2/447>

Codeheir. (n.d.). Flocking simulation [p5.js sketch]. p5.js Editor.

<https://editor.p5js.org/codeheir/sketches/73Fnq0QyW>

Codeheir. (2021). The flocking algorithm.

<https://codeheir.com/2021/03/27/the-flocking-algorithm/>

Compton, K. *Tracery: A text generation JS library*.

<http://www.galaxykate.com/pdfs/galaxykate-zinetracery.pdf>

Compton, K., Kybartas, B., & Mateas, M. (2015). Tracery: An Author-Focused Generative Text Tool. In H. Schoenau-Fog, L. E. Bruni, S. Louchart, & S. Baceviciute (Eds.), *Interactive Storytelling* (Vol. 9445, pp. 154–161). Springer International Publishing.

https://doi.org/10.1007/978-3-319-27036-4_14

Conway, J. H. (1970). *The Game of Life*. Scientific American, 223(4), 4.

Eickholt, J. (2018, August). *Barriers to Active Learning for Computer Science Faculty*. arXiv.

<http://arxiv.org/abs/1808.02426>

Eric-the-Geric. (n.d.). FinalProject [GitHub repository]. GitHub.

<https://github.com/Eric-the-Geric/FinalProject/tree/main>

Freeman, S., Eddy, S., McDonough, M., Michelle, S., Okoroafor, N., Jordt, H., & Pat Wenderoth, M. *Active learning increases student performance in science, engineering, and mathematics*. <https://doi.org/10.1073/pnas.1319030111>

Gamboa Maon, M. (2020). *Manolo Gamboa Naon*. Kate Vass Galerie.

<https://www.katevassgalerie.com/manolo-gamboa-naon>

GeeksforGeeks. (2017, June 24). *Turtle programming in Python*.

<https://www.geeksforgeeks.org/turtle-programming-python/>

GeeksforGeeks. (2021, October 19). *PyGame tutorial*.

<https://www.geeksforgeeks.org/pygame-tutorial/>

Google Colab. (2024). *Game of Life - Week 3* [Google Colab].

<https://colab.research.google.com/drive/1h24ZcWdXO-8hOrVFId29zvJEj5jQqVVv?usp=sharing>

Google Docs. (2024). *Finalized Creative Coding Course Syllabus* [Google Docs].

<https://docs.google.com/document/d/1LRjAQL4IEfjBUzALJSHoSVZFgSEIJt9-jEOOREzuDmM/edit?usp=sharing>

Golid, K. (ca. 2020). Kjetil Golid. Kate Vass Galerie.

<https://www.katevassgalerie.com/kjetil-golid>

Inkle Studios. *Writing web-based interactive fiction with ink*. Retrieved May 1, 2024, from

<https://www.inklestudios.com/ink/web-tutorial/>

Inkle Studios. (2022). *Running Your Ink*.

<https://github.com/inkle/ink/blob/master/Documentation/RunningYourInk.md>

Ivanitskaya, L., Clark, D., Montgomery, G., & Primeau, R. (2002). Interdisciplinary learning: Process and outcomes. *Innovative Higher Education*, 27(2), 95-111.

<https://link.springer.com/article/10.1007/s10755-002-2819-1>

John Horrigan. (2019). *Introduction to Context Free*.

<https://github.com/MtnViewJohn/context-free/wiki>

Kejriwal, M. (2022, August). *A Pedagogical Framework for Understanding the Alignment Between Classroom Project Evaluations and Real-World Industry Requirements*.

<https://peer.asee.org/a-pedagogical-framework-for-understanding-the-alignment-between-classroomproject-evaluations-and-real-world-industry-requirements>

- kipling. *CFDG Gallery - View a design*. Retrieved May 3, 2024, from <https://www.contextfreeart.org/gallery/view.php?id=3159>
- Kirilloff, G. (2021). Interactive Fiction in the Humanities Classroom: How to Create Interactive Text Games Using Twine. *The Programming Historian*, 10(10). <https://doi.org/10.46430/phen0095>
- Kogan, G. (2018). *ITP Fall 2018 - Class 02: Machine learning*. ml4a. <https://ml4a.github.io/classes/itp-F18/02/>
- Kogan, G. (2020). *Glow model example* [Jupyter notebook]. GitHub. <https://github.com/ml4a/ml4a/blob/master/examples/models/glow.ipynb>
- Kogan, G. (2021). *DeepDream model example* [Jupyter notebook]. GitHub. <https://github.com/ml4a/ml4a/blob/master/examples/models/deepdream.ipynb>
- Kogan, G. (n.d.). *Machine learning for artists*. Retrieved October 2, 2024, from <https://ml4a.github.io/ml4a/>
- Liao, Y.-C., & Ringler, M. (2023). Backward design: Integrating active learning into undergraduate computer science courses. *Cogent Education*, 10(1), 2204055–2204056. <https://doi.org/10.1080/2331186X.2023.2204055>
- Matplotlib Development Team. (n.d.). *Matplotlib documentation*. <https://matplotlib.org/stable/contents.html>
- MNIST handwritten digit database*, Yann LeCun, Corinna Cortes, and Chris Burges. (n.d.). Retrieved October 2, 2024, from <https://yann.lecun.com/exdb/mnist/>
- NumPy Developers. (n.d.). *NumPy documentation*. <https://numpy.org/doc/>
- Prince, M. (2004). Does active learning work? A review of the research. *Journal of Engineering Education*, 93(3), 223-231. <https://doi.org/10.1002/j.2168-9830.2004.tb00809.x>
- Processing Foundation. (n.d.-a). *Education*. <https://processingfoundation.org/education>
- Processing Foundation. (n.d.-b). *Libraries*. <https://p5js.org/libraries/>

- Processing Foundation. *Projects*. Retrieved May 1, 2024, from <https://processingfoundation.org/projects>
- Pygame Community. (n.d.). *Pygame documentation*. <https://www.pygame.org/docs/>
- Pygame Community. (n.d.). *Introduction to Pygame — Pygame tutorial 2019 documentation*. https://pygame.readthedocs.io/en/latest/1_intro/intro.html
- Python Software Foundation. (n.d.). *Turtle graphics for Python documentation*. <https://docs.python.org/3/library/turtle.html>
- Slackermanz. (2021). *Understanding multiple neighborhood cellular automata*. <https://slackermanz.com/understanding-multiple-neighborhood-cellular-automata/>
- Sweigart, A. (2010). *Chapter 2 — Pygame basics*. Retrieved October 2, 2024, from <https://inventwithpython.com/pygame/chapter2.html>
- Tran, A., Ong, C. S., & Wolf, C. (2018). Combining active learning suggestions. *Peerj Computer Science*, 4, e157. <https://doi.org/10.7717/peerj-cs.157>
- Turtle — Turtle graphics*. (n.d.). Python Documentation. Retrieved October 2, 2024, from <https://docs.python.org/3/library/turtle.html>
- Understanding Multiple Neighborhood Cellular Automata - Slackermanz*. (2021, May 23). <https://slackermanz.com/understanding-multiple-neighborhood-cellular-automata/>
- Vernon, D. T., & Blake, R. L. (1993). Does problem-based learning work? A meta-analysis of evaluative research. *Academic Medicine*, 68(7), 550–551. https://journals.lww.com/academicmedicine/abstract/1993/07000/does_problem_based_learning_work_a_meta_analysis.15.aspx
- Wong, T. (2008, September). Boids: Flocking behavior in artificial life. Stanford University. <https://cs.stanford.edu/people/eroberts/courses/soco/projects/2008-09/modeling-natural-systems/boids.html>

- Woo, K., & Falloon, G. (2022). Problem solved, but how? An exploratory study into students' problem solving processes in creative coding tasks. *Thinking Skills and Creativity*, 46, 101193–. <https://doi.org/10.1016/j.tsc.2022.101193>
- WPI Creative Coding Team. (2024, October 10). Course Resources.
<https://docs.google.com/spreadsheets/d/16e1Rf05Dw5olliFDvmGHIFTeK6QCapei8MU8bUcQZ68/edit?usp=sharing>
- Yang, H., & Zhang, L. (2016). Promoting Creative Computing: origin, scope, research and applications. *Digital Communications and Networks*, 2(2), 84–91.
<https://doi.org/10.1016/j.dcan.2016.02.001>
- Zhang, L. (2017). *Foundations of research in creative computing*.
<https://doi.org/10.17870/bathspa.00011588>

Appendix A. Creative Coding Interview Consent Agreement

Informed Consent Agreement for Participation in a Research Study

Investigators:

Artem Iurkovskyi, Ryan Wylie, Colin Hoar

Contact Information:

aiurkovskyi@wpi.edu, rawylie@wpi.edu, cmhoar@wpi.edu

Title of Research Study:

Designing a Creative Coding Course

Introduction:

You are being asked to participate in a research study. Before you agree, however, you must be fully informed about the purpose of the study, the procedures to be followed, and any benefits, risks, or discomfort that you may experience as a result of your participation. This form presents information about the study so that you may make a fully informed decision regarding your participation.

Purpose of the study:

The purpose of the study is to design a course in creative coding at the Zurich Institute of Applied Sciences. To achieve this, the Investigators want to better understand how creative coding is situated through interviewing educators.

Procedures to be followed:

You will be asked multiple open-ended questions about your experience with creative coding and personal perspectives on it. This conversation will be conducted in the form of a semi-structured interview where an Investigator will be able to ask additional questions related to the main topics outlined below. A participation of 45 minutes in duration is expected but not required.

Interview Questions:

- Creative Coding Experience: What experience do you have with creative coding, and what tools have you used that relate to this field?
- The appeal of Creative Coding: What about creative coding stands out to you?
- Intersection With Art: How does creative coding intersect with art and design? Can you provide examples of how it's utilized in these fields?
- Personal Projects: Could you share a project you've worked on or admired that exemplifies the potential of creative coding?

Benefits to research participants and others:

There are no direct expected benefits to the research participants or others.

Record keeping and confidentiality:

The interview will be audio recorded and transcribed during the 7 days after the interview. The transcribed data will be de-identified and stored in a Google Drive folder accessible only to the Investigators. The original audio recording will be deleted after the interview text has been transcribed. Records of your participation in this study will be held confidential so far as permitted by law. However, the study investigators, the sponsor, or its designee, and, under certain circumstances, the Worcester Polytechnic Institute Institutional Review Board (WPI IRB) will be able to inspect and have access to confidential data that identify you by name. Any publication or presentation of the data will not identify you.

Compensation or treatment in the event of injury:

The research does not involve the risk of injury or harm. No compensation to the Study Participant will be granted in case of injury. You do not give up any of your legal rights by signing this statement.

For more information about this research or about the rights of research participants, or in case of research-related injury, contact:

Investigator (Artem Iurkovskyi, Tel. 646 715-8184, Email: aiurkovskyi@wpi.edu)

Investigator (Ryan Wylie, Email: rawylie@wpi.edu)

Investigator (Colin Hoar, Tel. 978 831-9754, Email: cmhoar@wpi.edu)

IRB Manager (Ruth McKeogh, Tel. 508 831-6699, Email: irb@wpi.edu) and the Human Protection Administrator (Gabriel Johnson, Tel. 508-831-4989, Email: gjohnson@wpi.edu)

Your participation in this research is voluntary. Your refusal to participate will not result in any penalty to you or any loss of benefits to which you may otherwise be entitled. You may decide to stop participating in the research at any time without penalty or loss of other benefits. The project investigators retain the right to cancel or postpone the experimental procedures at any time they see fit.

By signing below, you acknowledge that you have been informed about and consent to be a participant in the study described above. Make sure that your questions are answered to your satisfaction before signing. You are entitled to retain a copy of this consent agreement.

Study Participant Signature:

Date Signed:

Study Participant Name (Please print):

Signature of the Person who explained this study:

Date Signed:

Appendix B. Teaching Strategy Interview Consent Agreement

Informed Consent Agreement for Participation in a Research Study

Investigators:

Artem Iurkovskyi, Ryan Wylie, Colin Hoar

Contact Information:

aiurkovskyi@wpi.edu, rawylie@wpi.edu, cmhoar@wpi.edu

Title of Research Study:

Designing a Creative Coding Course

Introduction:

You are being asked to participate in a research study. Before you agree, however, you must be fully informed about the purpose of the study, the procedures to be followed, and any benefits, risks, or discomfort that you may experience as a result of your participation. This form presents information about the study so that you may make a fully informed decision regarding your participation.

Purpose of the study:

The purpose of the study is to design a course in creative coding at the Zurich Institute of Applied Sciences. To achieve this, the Investigators want to evaluate the effectiveness of different teaching strategies by interviewing educators and students.

Procedures to be followed:

You will be asked multiple open-ended questions about your teaching experience and the strategies that you find effective. This conversation will be conducted in the form of a semi-structured interview where an Investigator will be able to ask additional questions related to the main topics outlined below. A participation of 45 minutes in duration is expected but not required.

Interview Questions:

- Experience in Course Design: Can you describe your process for designing a new course syllabus, particularly focusing on any innovative strategies you have employed?
- Implementation of Innovations: What innovative teaching strategies have you recently implemented in your courses, and what motivated these changes?
- Feedback Collection: How do you collect and integrate student feedback regarding the course design and teaching strategies?

- Assessment of Learning Outcomes: How do you assess whether the teaching strategies you've implemented are effective in enhancing student learning?
- Recommendations for Peers: What advice would you give to other educators looking to innovate their course syllabi and teaching methods?

Benefits to research participants and others:

There are no direct expected benefits to the research participants or others.

Record keeping and confidentiality:

The interview will be audio recorded and transcribed during the 7 days after the interview. The transcribed data will be de-identified and stored in a Google Drive folder accessible only to the Investigators. The original audio recording will be deleted after the interview text has been transcribed. Records of your participation in this study will be held confidential so far as permitted by law. However, the study investigators, the sponsor, or its designee, and, under certain circumstances, the Worcester Polytechnic Institute Institutional Review Board (WPI IRB) will be able to inspect and have access to confidential data that identify you by name. Any publication or presentation of the data will not identify you.

Compensation or treatment in the event of injury:

The research does not involve the risk of injury or harm. No compensation to the Study Participant will be granted in case of injury. You do not give up any of your legal rights by signing this statement.

For more information about this research or about the rights of research participants, or in case of research-related injury, contact:

Investigator (Artem Iurkovskyi, Tel. 646 715-8184, Email: aiurkovskyi@wpi.edu)

Investigator (Ryan Wylie, Email: rawylie@wpi.edu)

Investigator (Colin Hoar, Tel. 978 831-9754, Email: cmhoar@wpi.edu)

IRB Manager (Ruth McKeogh, Tel. 508 831-6699, Email: irb@wpi.edu) and the Human Protection Administrator (Gabriel Johnson, Tel. 508-831-4989, Email: gjohnson@wpi.edu)

Your participation in this research is voluntary. Your refusal to participate will not result in any penalty to you or any loss of benefits to which you may otherwise be entitled. You may decide to stop participating in the research at any time without penalty or loss of other benefits. The project investigators retain the right to cancel or postpone the experimental procedures at any time they see fit.

By signing below, you acknowledge that you have been informed about and consent to be a participant in the study described above. Make sure that your questions are answered to your satisfaction before signing. You are entitled to retain a copy of this consent agreement.

Study Participant Signature:

Date Signed:

Study Participant Name (Please print):

Signature of the Person who explained this study:

Date Signed:

Appendix C. Student Survey

Investigators:

Artem Iurkovskyi, Ryan Wylie, Colin Hoar

Contact Information:

aiurkovskyi@wpi.edu, rawylie@wpi.edu, cmhoar@wpi.edu

Title of Research Study:

Designing a Creative Coding Course

Introduction:

You are being asked to participate in a research study. Before you agree, however, you must be fully informed about the purpose of the study, the procedures to be followed, and any benefits, risks, or discomfort that you may experience as a result of your participation. This form presents information about the study so that you may make a fully informed decision regarding your participation.

- What is your major?
- Would you be interested in taking a continuing education course?
- On a scale of 1 to 10, how would you rate your current knowledge of what creative coding is?
- On a scale of 1 to 10, how interested would you be in taking a creative coding course?

Course Description

Unleash your creativity and transform code into art in this eight-week course about creative coding. Students with basic programming experience who want to explore artistic expression or enhance their programming skills, specifically with Python, are welcome to join.

Students will learn about exciting topics such as Conway's Game of Life, cellular automaton simulating intricate patterns, and Machine Learning, where algorithms learn to create art. Through a flipped classroom environment, students will engage with core concepts on their own time with pre-lecture videos, allowing students hands-on sessions during lectures to encourage exploration of creativity.

Students will create multiple visual art projects throughout the course, culminating in a final project where they can choose what they want to make based on their interests. At the end of the course, students will leave with deepened programming skills and an appreciation for creativity, proven by the digital portfolio they have created.

- What aspects of creative coding interest you the most?
(Select all that apply)
- A. Creating visual art through programming
- B. Exploring the intersection of technology and creativity

- C. Learning new programming techniques in a creative context
 - D. Using code to simulate natural processes and patterns
 - E. Experimenting with generative art and algorithms
 - F. Other (please specify)
- What challenges do you think you might face when taking a creative coding course? (*For example, time commitment, difficulty of programming, creative blocks, etc.*)
 - What questions or concerns do you have about the creative coding course? (*Is there anything you would like to clarify or are unsure about?*)
 - How would you prefer to learn about creative coding topics?
 - A. Hands-on coding exercises during class
 - B. Independent exploration with guided materials
 - C. Collaboration with peers on creative projects
 - D. Instructor-led lectures
 - E. A mix of all of the above
 - F. Other (please specify)
 - What do you hope to take away from a course like this? (*E.g., an understanding of creative coding concepts, improved programming skills, a digital portfolio, etc.*)
 - On a scale of 1 to 10, how likely are you to enroll in this course if it becomes available? (*1 = Not likely at all, 10 = Very likely*)

Appendix D. Effectiveness Interview Consent Agreement

Informed Consent Agreement for Participation in a Research Study

Investigators:

Artem Iurkovskyi, Ryan Wylie, Colin Hoar

Contact Information:

aiurkovskyi@wpi.edu, rawylie@wpi.edu, cmhoar@wpi.edu

Title of Research Study:

Designing a Creative Coding Course

Introduction:

You are being asked to participate in a research study. Before you agree, however, you must be fully informed about the purpose of the study, the procedures to be followed, and any benefits, risks, or discomfort that you may experience as a result of your participation. This form presents information about the study so that you may make a fully informed decision regarding your participation.

Purpose of the study:

The purpose of the study is to design a course in creative coding at the Zurich Institute of Applied Sciences. To achieve this, the Investigators want to evaluate the quality and effectiveness of the finalized course syllabus.

Procedures to be followed:

You will be asked multiple open-ended questions regarding the quality and effectiveness of our course syllabus. This conversation will be conducted in the form of a semi-structured interview where an Investigator will be able to ask additional questions related to the main topics outlined below. A participation of 45 minutes in duration is expected but not required.

Interview Questions:

- Effectiveness of Course Structure: How well do you think the course structure aligns with the learning outcomes, and are there any adjustments you would recommend to improve this alignment?
- Effectiveness of Course Schedule: How do you feel about the pacing of the content delivery? Are there areas that might benefit from either more depth or more concise coverage?

- Effectiveness of Teaching Strategies: What are your thoughts on the teaching methods suggested in the syllabus? Are there any additional strategies that could improve student engagement and comprehension?
- Overall Effectiveness: Based on your experience, do you foresee any challenges with the proposed schedule or learning materials that might affect student success or course flow?

Benefits to research participants and others:

There are no direct expected benefits to the research participants or others.

Record keeping and confidentiality:

The interview will be audio recorded and transcribed during the 7 days after the interview. The transcribed data will be de-identified and stored in a Google Drive folder accessible only to the Investigators. The original audio recording will be deleted after the interview text has been transcribed. Records of your participation in this study will be held confidential so far as permitted by law. However, the study investigators, the sponsor, or its designee, and, under certain circumstances, the Worcester Polytechnic Institute Institutional Review Board (WPI IRB) will be able to inspect and have access to confidential data that identify you by name. Any publication or presentation of the data will not identify you.

Compensation or treatment in the event of injury:

The research does not involve the risk of injury or harm. No compensation to the Study Participant will be granted in case of injury. You do not give up any of your legal rights by signing this statement.

For more information about this research or about the rights of research participants, or in case of research-related injury, contact:

Investigator (Artem Iurkovskiy, Tel. 646 715-8184, Email: aiurkovskiy@wpi.edu)

Investigator (Ryan Wylie, Email: rawylie@wpi.edu)

Investigator (Colin Hoar, Tel. 978 831-9754, Email: cmhoar@wpi.edu)

IRB Manager (Ruth McKeogh, Tel. 508 831-6699, Email: irb@wpi.edu) and the Human Protection Administrator (Gabriel Johnson, Tel. 508-831-4989, Email: gjohnson@wpi.edu)

Your participation in this research is voluntary. Your refusal to participate will not result in any penalty to you or any loss of benefits to which you may otherwise be entitled. You may decide to stop participating in the research at any time without penalty or loss of other benefits.

The project investigators retain the right to cancel or postpone the experimental procedures at any time they see fit.

By signing below, you acknowledge that you have been informed about and consent to be a participant in the study described above. Make sure that your questions are answered to your satisfaction before signing. You are entitled to retain a copy of this consent agreement.

Study Participant Signature:

Date Signed:

Study Participant Name (Please print):

Signature of the Person who explained this study:

Date Signed:

Appendix E. Google Colab Assignment

This appendix contains snippets of Google Colab assignments with intentionally uncompleted code. The students' job is to complete the code where there is a dashed line and then run the whole function. This is done so that the students proceed fast to the creative part of the module and at the same time understand the fundamentals of the program they are running. Correctly completed code in the first assignment renders a video of a simulated Conway's Game of Life, the most standard cellular automaton. In a similar style of uncompleted code, students are introduced to Multiple Neighborhood Cellular Automata (MNCA) in the second assignment. This lets students generate creative visual output and shows the possibilities for experimentation. The assignment with code completion task can be accessed by following this link: [Google Colab, Uncompleted Assignment](#). The completed code for instructors will be provided per their request at aiurkovskyi[at]wpi.edu.

✓ Conway's Game of Life

Conway's Game of Life is a famous example of cellular automaton—a type of simulation involving cells that evolve based on simple rules. Developed by John Conway in 1970, this "game" doesn't involve any players; rather, it is a zero-player game where the outcome is determined by an initial state and the defined rules of the system. The grid evolves in discrete steps, and each cell follows straightforward rules of life and death.

The Game of Life operates on a grid of cells, where each cell can either be alive (1) or dead (0). The state of the grid evolves over time according to these rules:

1. Any live cell with fewer than two live neighbors dies (underpopulation).
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies (overpopulation).
4. Any dead cell with exactly three live neighbors becomes alive (reproduction).

These simple rules lead to complex and often beautiful patterns, some of which are stable, oscillate, or even move across the grid.

✓ Assignment 1: Conway's Game of Life

In this assignment, you will implement Conway's Game of Life, step-by-step. Some parts of the code have been left blank—your job is to fill them in to complete the simulation.

```
# Import necessary libraries
import numpy as np
import pygame
import cv2

# Step 1.1.1: Create the GameOfLife class with an initialization function
class GameOfLife:
    def __init__(self, size, scale=1):
        """
        Initialize the Game of Life.

        Parameters:
        - size: Tuple indicating the size of the grid (rows, cols).
        - scale: Scaling factor for display (default is 1).
        """
        self.size = size
        self.scale = scale
        # TODO: Step 1.1.1.1 - Initialize a random grid of 0s and 1s. Fill in the code below.
        self.grid = _____ # Use np.random.choice to create a grid of 0s and 1s

# Step 1.1.2: Define the step function to update the grid
def step(self):
    """
    Perform one step in the Game of Life.
    Update each cell based on its neighbors.
    """
    # Calculate the sum of neighbors for each cell using np.roll
    neighbors = (
        np.roll(np.roll(self.grid, 1, 0), 1, 1) + # top-left
        np.roll(np.roll(self.grid, 1, 0), 0, 1) + # top
        np.roll(np.roll(self.grid, 1, 0), -1, 1) + # top-right
        np.roll(np.roll(self.grid, 0, 0), 1, 1) + # left
        np.roll(np.roll(self.grid, 0, 0), -1, 1) + # right
        np.roll(np.roll(self.grid, -1, 0), 1, 1) + # bottom-left
        np.roll(np.roll(self.grid, -1, 0), 0, 1) + # bottom
        np.roll(np.roll(self.grid, -1, 0), -1, 1) # bottom-right
    )

    # TODO: Step 1.1.2.1 - Update the grid based on the rules of Conway's Game of Life
    # Hint: Use np.where to apply the rules to the grid
    self.grid = np.where(_____, 1, 0)

# Step 1.1.3: Define the render function
def render(self, screen, scale=10):
    """
```

Figure E.1: Game of Life; Google Colab Notebook Snippet 1 ([Google Colab, 2024](#))

```

Parameters:
- screen: Pygame screen object.
- scale: Scaling factor to magnify the grid for visualization.
"""
# TODO: Step 1.1.3.1 - Scale the grid to fit the display using np.kron
scaled_data = np.kron(self.grid * 255, _____) # Magnify by the scale factor

# Convert the scaled grid to a Pygame surface and display it
surf = pygame.surfarray.make_surface(np.stack([scaled_data] * 3, axis=-1))
screen.fill((0, 0, 0))
screen.blit(surf, (0, 0))
pygame.display.flip()

# Step 1.1.4: Write the main function to run the simulation and save as a video
# TODO: Step 1.1.4 - Initialize world size, screen size, scale, the number of steps, and set video name variable
def run_simulation_and_save_video(_____):
    pygame.init()
    screen = pygame.display.set_mode(screen_size)
    game = GameOfLife(world_size)

    # Video writer setup
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    fps = 10
    video_writer = cv2.VideoWriter(video_name, fourcc, fps, screen_size)

    # TODO: Step 1.1.4.1 - Run the simulation loop to evolve the grid and record frames
    for _ in range(steps):
        game.step()
        game.render(screen, scale)

        # Capture the screen and save to video
        frame = pygame.surfarray.array3d(screen)
        frame = np.transpose(frame, (1, 0, 2))
        video_writer.write(cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))

    pygame.time.delay(100)

    video_writer.release()
    pygame.quit()

# Step 1.1.5: Execute the simulation
# TODO: Uncomment the line below to run the simulation once your class is complete
# run_simulation_and_save_video()

```

Assignment 2: MNCA Game of Life

✓ Introduction to MNCA (Multiple Neighborhood Cellular Automata)

In Assignment 1, you worked on implementing Conway's Game of Life, which uses simple rules based on the state of a cell's neighbors to determine its next state. In this assignment, you will implement a more complex variation called Multiple Neighborhood Cellular Automata (MNCA).

MNCA allows us to consider a much wider range of possible interactions within a cell's neighborhood by utilizing different kernels (patterns used for convolution). These kernels define how cells influence each other, enabling us to simulate more intricate behaviors compared to the simpler rules of Conway's Game of Life.

If you want to learn more about MNCA, including detailed explanations and mathematical insights, visit [this website](#). This resource provides in-depth information about MNCA, including examples of rules and generated worlds.

```

# Step 2.1.0: Import necessary libraries
# TODO: Step 2.1.0.1 - Import the required libraries
import _____ as np
import _____
from _____ import fftconvolve # Use FFT convolution for speed
import _____

```

Figure E.2: Game of Life; Google Colab Notebook Snippet 2 ([Google Colab, 2024](#))

```

import sys

# Step 2.1.1: Define the function to generate a circular kernel
def generate_circular_kernel(radius):
    """
    Generate a circular kernel used in MNCA.

    Parameters:
    - radius: Radius of the circular kernel.

    Returns:
    - kernel: A 2D numpy array representing the circular kernel.
    """
    # TODO: Step 2.1.1.1 - Complete the kernel generation
    size = 2 * radius + 1
    y, x = np.ogrid[-radius:radius+1, -radius:radius+1]
    mask = _____ # Fill in the condition to create a circular mask
    kernel = np.zeros((size, size))
    kernel[mask] = _____ # Fill in the value to apply to the circular area
    return kernel

# Step 2.1.2: Define the MNCA function to update the grid state
def MNCA_Test(input_buffer):
    """
    Perform a single step in MNCA based on kernel convolutions.

    Parameters:
    - input_buffer: The current state of the grid.

    Returns:
    - clamped_result: The updated state of the grid.
    """
    # TODO: Step 2.1.2.1 - Generate two circular kernels of different sizes
    kernel1 = generate_circular_kernel(_____) # Replace with an appropriate value for radius
    kernel2 = generate_circular_kernel(_____) # Replace with an appropriate value for radius

    # TODO: Step 2.1.2.2 - Apply FFT convolution to the input buffer
    result1 = fftconvolve(input_buffer, kernel1 / np.sum(kernel1), mode='_____') # Fill in the mode
    result2 = fftconvolve(input_buffer, kernel2 / np.sum(kernel2), mode='_____') # Fill in the mode

    # TODO: Step 2.1.2.3 - Update the grid state based on different conditions
    condition3 = (result2 > 0.12) & (result2 < _____) # Fill in the upper bound value
    update3 = np.where(condition3, input_buffer - 0.14, input_buffer)

    condition1 = (result1 > _____) & (result1 < 0.34) # Fill in the lower bound value
    update1 = np.where(condition1, update3 + _____, update3 - 0.02) # Fill in the value to add

    condition2 = (result1 > 0.32) & (result1 < _____) # Fill in the upper bound value
    update2 = np.where(condition2, update1 + 0.08, update1 - _____) # Fill in the value to subtract

    # Clamp the result to keep all values within [0, 1]
    clamped_result = np.clip(_____, 0.0, 1.0) # Fill in the variable to clamp
    return clamped_result

# Step 2.1.3: Define the World class that manages the cellular automaton
class World:
    def __init__(self, size, scale=1, dtype=np.float16, channels=1):
        """
        Initialize the World for MNCA.

        Parameters:
        - size: Tuple indicating the size of the grid (rows, cols).
        - scale: Scaling factor for display.
        - dtype: Data type for the grid buffer.
        - channels: Number of channels (default is 1).
        """
        self.size = size
        self.scale = scale
        self.dtype = dtype
        self.channels = channels
        scaled_size = (size[0] // scale, size[1] // scale)
        # TODO: Step 2.1.3.1 - Define the buffer shape
        buffer_shape = scaled_size if channels == 1 else (_____,) # Fill in the correct buffer shape
        self.buffer1 = np.zeros(buffer_shape, dtype=dtype)
        self.buffer2 = np.zeros(buffer_shape, dtype=dtype)

    def seed(self):

```

Figure E.3: Game of Life; Google Colab Notebook Snippet 3 ([Google Colab, 2024](#))

```

"""
Seed the initial grid with random values.
"""
# TODO: Step 2.1.3.2 - Seed the initial buffer with appropriate values
if self.channels == _____: # Fill in the appropriate condition for channels
    if np.issubdtype(self.dtype, np.integer):
        self.buffer1 = np.random.randint(0, 256, self.size, dtype=self.dtype)
    elif np.issubdtype(self.dtype, np.floating):
        self.buffer1 = np.random.random(self.size).astype(self.dtype)
    elif np.issubdtype(self.dtype, np.bool_):
        self.buffer1 = np.random.choice([True, False], self.size)
    else:
        raise ValueError("Unsupported data type for buffer.")
else:
    if np.issubdtype(self.dtype, np.integer):
        self.buffer1 = np.random.randint(0, 256, (_____), dtype=self.dtype) # Fill in the correct size
    elif np.issubdtype(self.dtype, np.floating):
        self.buffer1 = np.random.random((_____)).astype(self.dtype) # Fill in the correct size
    else:
        raise ValueError("Unsupported data type for RGB buffer.")

def step(self):
    """
    Perform a single step in the simulation by updating the buffer.
    """
    # TODO: Step 2.1.3.3 - Update the buffer using MNCA_Test function
    self.buffer2 = MNCA_Test(_____) # Fill in the correct input
    self.buffer1, self.buffer2 = _____, _____ # Swap the buffers

def get_state(self):
    """
    Get the current state of the grid.
    """
    # TODO: Step 2.1.3.4 - Return the correct buffer state
    return _____

def render(self, screen, scale=10):
    """
    Render the current state of the grid using Pygame.

    Parameters:
    - screen: Pygame screen object.
    - scale: Scaling factor to magnify the grid.
    """
    state_data = self.get_state()

    # Normalize and scale the data for rendering (Pygame expects values in range 0-255)
    # TODO: Step 2.1.3.5 - Convert the state data to the appropriate range for rendering
    scaled_data = (state_data * _____).astype(np.uint8) # Fill in the value to scale

    # If grayscale, we need to convert it to RGB for Pygame rendering
    if len(scaled_data.shape) == 2:
        scaled_data = np.stack([scaled_data] * _____, axis=-1) # Fill in the number of channels

    # Scale up the image for better visibility (zoom in)
    scaled_data = np.kron(scaled_data, np.ones((scale, scale, _____))) # Fill in the correct value

    # Center the simulation on the screen
    screen_center = (screen.get_width() // 2, screen.get_height() // 2)
    scaled_size = scaled_data.shape[:2]
    top_left = (screen_center[0] - scaled_size[1] // 2, screen_center[1] - scaled_size[0] // 2)

    # Create a surface for the scaled data and blit it to the screen
    surf = pygame.surfarray.make_surface(scaled_data)
    screen.fill((0, 0, 0)) # Clear the screen
    screen.blit(surf, top_left)
    pygame.display.flip()

# Step 2.1.4: Define the main function to run the simulation and save as a video
def run_simulation_and_save_video(world_size=(1000, 1000), screen_size=(1000, 1000), scale=5, steps=100, video_name='mnca_simulation.mp4'):
    """
    Run the MNCA simulation and save it as a video.

    Parameters:
    - world_size: Size of the grid.
    - screen_size: Size of the screen.

```

Figure E.4: Game of Life; Google Colab Notebook Snippet 4 ([Google Colab, 2024](#))

```

- scale: Scaling factor for display.
- steps: Number of simulation steps.
- video_name: Name of the output video file.
"""
pygame.init()
screen = pygame.display.set_mode(screen_size)
world = World(world_size, scale)
world.seed()

# Video writer setup
# TODO: Step 2.1.4.1 - Set up the video writer
fourcc = cv2.VideoWriter_fourcc('*_____') # Fill in the codec
fps = _____ # Fill in the frames per second
video_writer = cv2.VideoWriter(video_name, fourcc, fps, screen_size)

for _ in range(steps):
    world.step()
    world.render(screen, scale)

    # Capture screen and convert it for video writing
    frame = pygame.surfarray.array3d(screen)
    frame = np.transpose(frame, (1, 0, _____)) # Fill in the correct axes order
    video_writer.write(cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))

    pygame.time.delay(_____) # Fill in the delay time

video_writer.release()
pygame.quit()

# Step 2.1.5: Execute the simulation
# TODO: Uncomment the line below to run the simulation once your class is complete
# run_simulation_and_save_video()

```

Figure E.5: Game of Life; Google Colab Notebook Snippet 5 ([Google Colab, 2024](#))

Appendix F. Interview Codebook

Codebook used for interview notes and syllabus notes analysis:

Research Objectives	Code	Description	Notes
1	Creative Coding	Basic information relating to creative coding	Added 9/3 by RW
	Tools	General creative coding tools	Added 9/3 by RW
	Course Resources	Resources about or used for a creative coding course (Languages, Applications, References)	Added 8/28 by RW Updated 9/3 to be more specific Moved to Obj 1 from Obj 3 9/9 to reflect methodology revisions
	Course Goals	Overall goals of the course: what a student should be able to learn from the course	Added 8/28 by RW Moved to Obj 1 from Obj 3 9/9 to reflect methodology revisions
	Projects	Descriptions of projects	Added 8/28 by RW Moved to Obj 1 from Obj 3 9/9 to reflect methodology revisions
	Course Topics	Examples of interesting topics to delve into, creative or programming-related	Added 8/28 by RW Updated 9/9 to be more specific Moved to Obj 1 from Obj 3 9/9 to reflect methodology revisions
	Course Structure	Information about structuring the course (Lectures, length of course, assignments)	Added 8/28 by RW Moved to Obj 1 from Obj 3 9/9 to reflect methodology revisions
	Audience	Information about the students who take the course	Added 8/29 by RW Moved to Obj 1 from Obj 3 9/9 to reflect methodology

			revisions
2	Teaching Strategies	General description of effective teaching strategies	Added 9/3 by RW
	Professor Perspective	Thoughts about teaching strategies from a teaching perspective	Added 9/3 by RW
	Student Perspective	Thoughts about teaching strategies from a learning perspective	Added 9/3 by RW
3	Familiar Exemplars	Examples of interesting topics for creative prompts that Swiss STEM students would find familiar	Added 8/28 by RW Updated 9/9 to be more specific
4	Interest	Specific aspects of the syllabus that students or professors found interesting	Added 9/9 by RW
	Effective	Specific aspects of the syllabus that professors found effective at teaching creative coding	Added 9/9 by RW
	Positive Feedback	Feedback where the person generally agrees with the syllabus	Added 9/9 by RW
	Negative Feedback	Feedback where the person disagrees with the syllabus and possibly offers alternative solutions	Added 9/9 by RW